

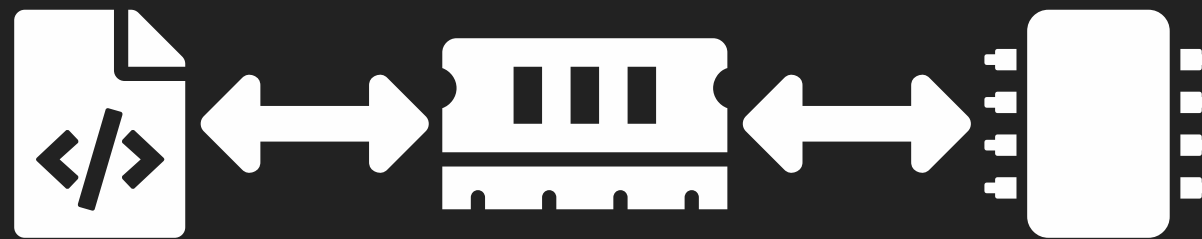
BETRIEBSSYSTEME

Technische Hochschule Mittelhessen

Andre Rein

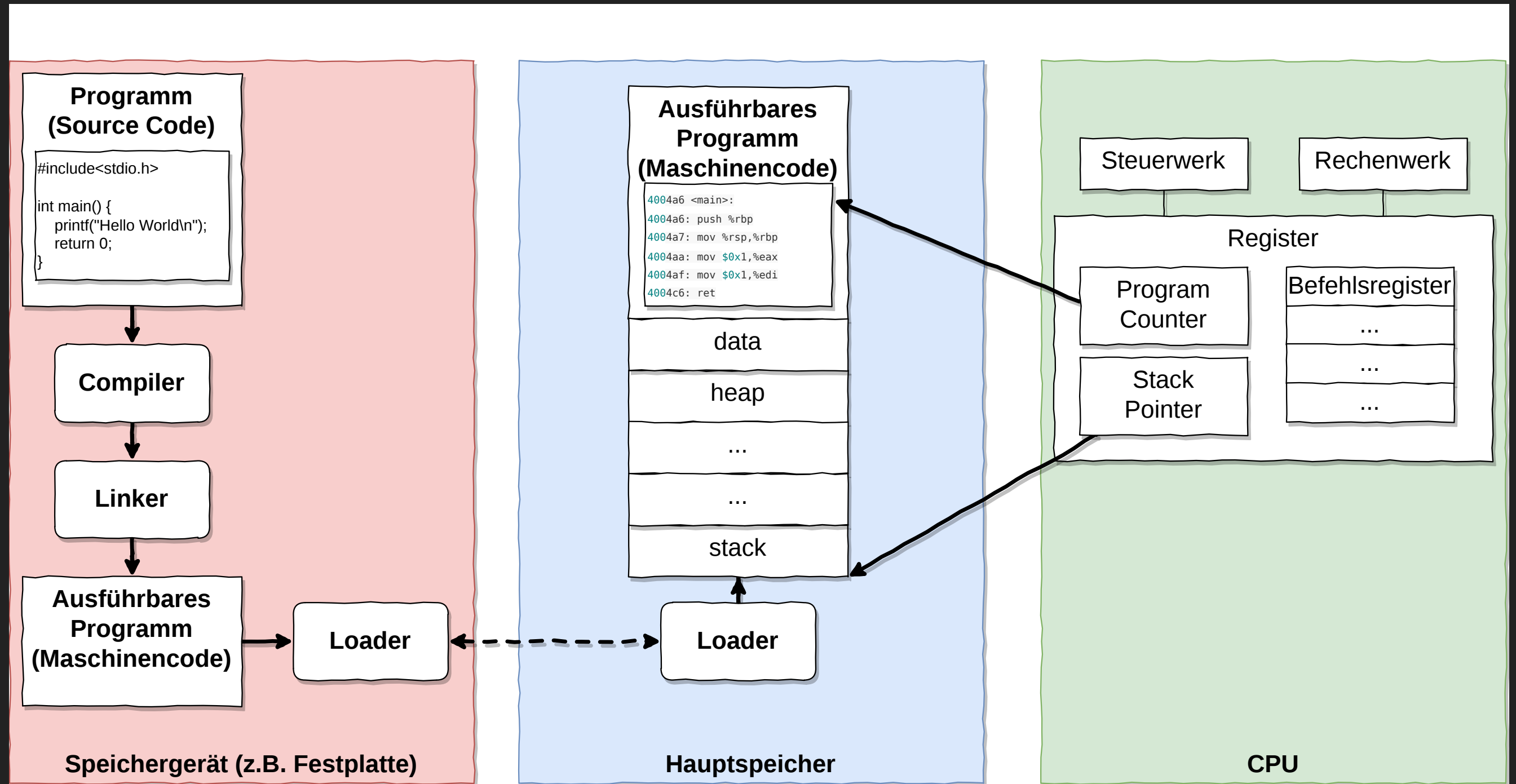
– Betriebssysteme Einführung –

WIE FUNKTIONIERT EIN COMPUTER?



???

WIE FUNKTIONIERT EIN COMPUTER?



WAS IST EIN BETRIEBSSYSTEM?

Ein **Betriebssystem** ist in aller erster Linie ein komplexes **Programm** das als **Vermittler** zwischen einem **Benutzer** eines Computers und der **Computerhardware** fungiert.

- Ziele eines Betriebssystems:
 - Die Computerhardware effizient zu nutzen
 - Die Ausführung von (mehreren) Benutzerprogrammen zu erlauben
 - Die Verwendung eines Computersystems benutzerfreundlich zu machen

WAS IST EIN BETRIEBSSYSTEM?

- Abstrakter Rechner 
 - Überdeckung der Eigenheiten der Rechnerplattform - Ermöglicht eine plattformunabhängige Nutzung
 - Vom IoT Device bis Supercomputer
- Verwaltung der Hardware und aller Peripheriegeräte   
 - Ein Benutzer und **Programmierer** muss sich nicht um die Eigenheiten der Hardware und Peripherie kümmern
 - Das schließt auch konkurrierende Zugriffe auf die Hardware ein
- Sammlung von Dienstprogrammen/Systemprogrammen — kopieren, sortieren, editieren, compilieren
- Sammlung von Kommunikationsmechanismen — Implementiert und bietet Schnittstellen zu unterschiedlichsten Rechnernetzen

WAS IST EIN BETRIEBSSYSTEM?

- Ein Betriebssystem ist Ressourcenzuweiser und -verteiler:
 - Es verwaltet alle Ressourcen auf einem Computer
 - Es entscheidet zwischen gegensätzlichen Anfragen um eine **effiziente** und **faire Ressourcennutzung** zu bieten
- Ein Betriebssystem ist ein Steuerungsprogramm:
 - Es steuert die Ausführung von Programmen
 - Es versucht Fehler und unsachgemäßen Gebrauch des Computers zu vermeiden zu behandeln

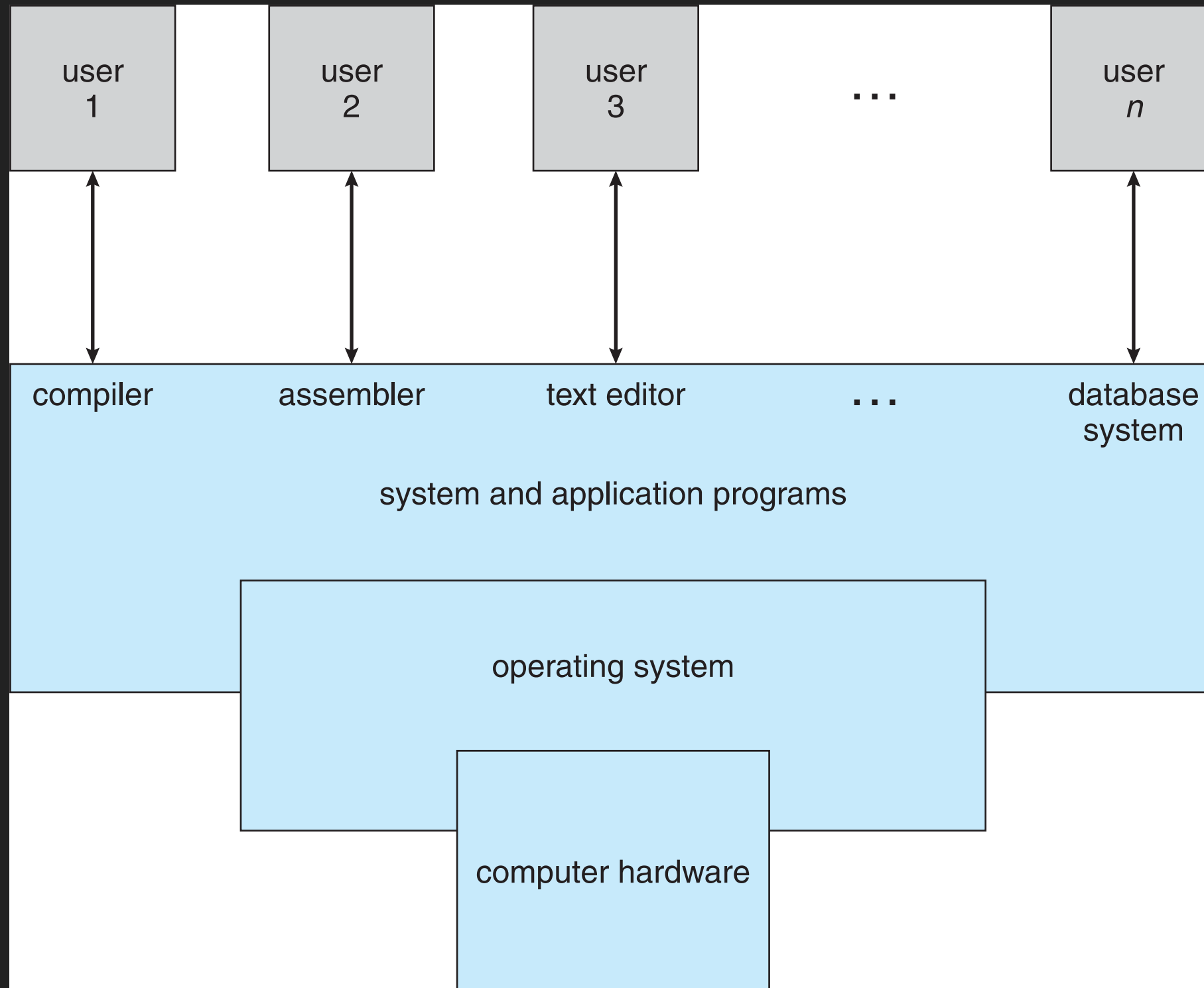
DEFINITION EINES BETRIEBSSYSTEMS ?

Es gibt keine allgemein akzeptierte Definition eines Betriebssystems

"Alles, was ein Hersteller liefert/anbietet, wenn Sie ein Betriebssystem bestellen oder herunterladen" ist eine gute Annäherung, aber es gibt sehr große Unterschiede.

- Ein fixer Bestandteil von jedem Betriebssystem ist der **Kernel**
 - Ein spezielles Programm das immer und ständig auf dem Computer ausgeführt wird
- Alles andere ist entweder:
 - ein Systemprogramm (wird oft mit dem Betriebssystem ausgeliefert), oder
 - ein Anwendungsprogramm

COMPUTERSYSTEME



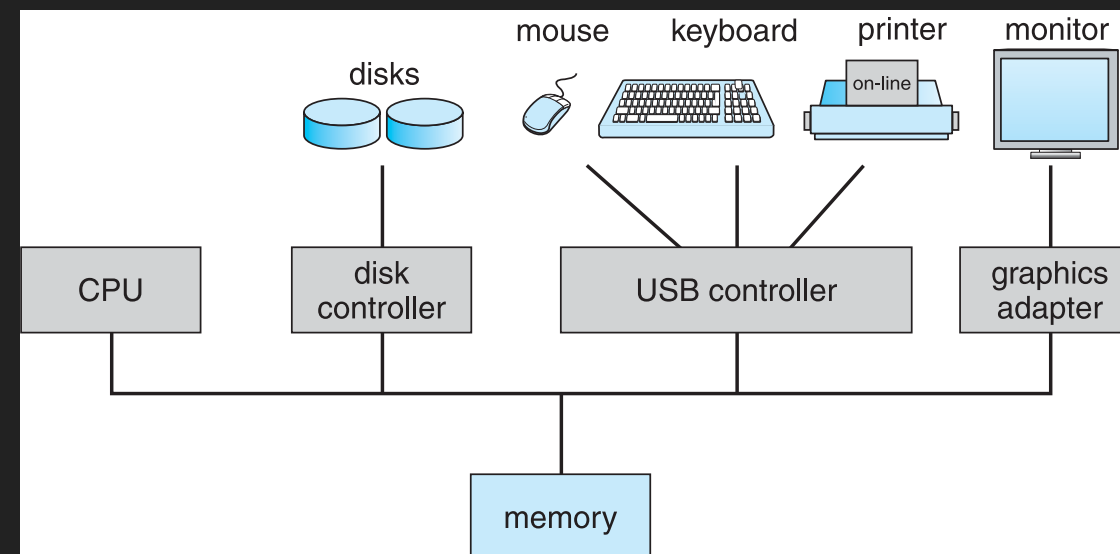
COMPUTERSYSTEME

Computersysteme können grob in vier Komponenten unterteilt werden:

- **Hardware** - bildet die grundlegenden Computerressourcen
 - CPU, Speicher, I/O-Geräte
- **Betriebssystem** - Steuert und koordiniert den Einsatz von Hardware zwischen verschiedenen Anwendungen und Benutzern
- **Anwendungsprogramme** - Löst Rechenprobleme der Benutzer und definiert die Art und Weise, wie die Systemressourcen dazu genutzt werden
 - Textverarbeitungsprogramme, Compiler, Webbrowser, Datenbanksysteme, Videospiele
- **Benutzer**: Menschen, Maschinen, andere Computer

BETRIEB EINES COMPUTERSYSTEMS

- Eine oder mehrere CPUs und Geräte (device controllers) verbinden sich über einen gemeinsamen Bus
 - Das ermöglicht den Zugriff auf den gemeinsamen Speicher
- Paralleler Betrieb aller Komponenten:
 - Alle Komponenten konkurrieren um Zugriff auf den Speicher



BETRIEB EINES COMPUTERSYSTEMS

- Input/Output (I/O)-Geräte und die CPU werden gleichzeitig ausgeführt
 - Jeder Gerätecontroller ist für einen bestimmten Gerätetyp zuständig
 - Jeder Gerätecontroller verfügt über einen lokalen Puffer
- Die CPU verschiebt Daten vom/zum Hauptspeicher in/vom lokalen Puffer
 - (*Ausnahme bei DMA (Direct Memory Access)*)
 - I/O ist hierbei immer erst vom Gerät zum lokalen Puffer des Controllers
- Der Gerätecontroller informiert die CPU, dass er seine Transfer beendet hat, indem er einen Interrupt (Hardware Interrupt) auslöst

Ein Interrupt kann zu einer beliebigen Zeit auftreten und ist somit
asynchron

INTERRUPT FUNKTIONEN

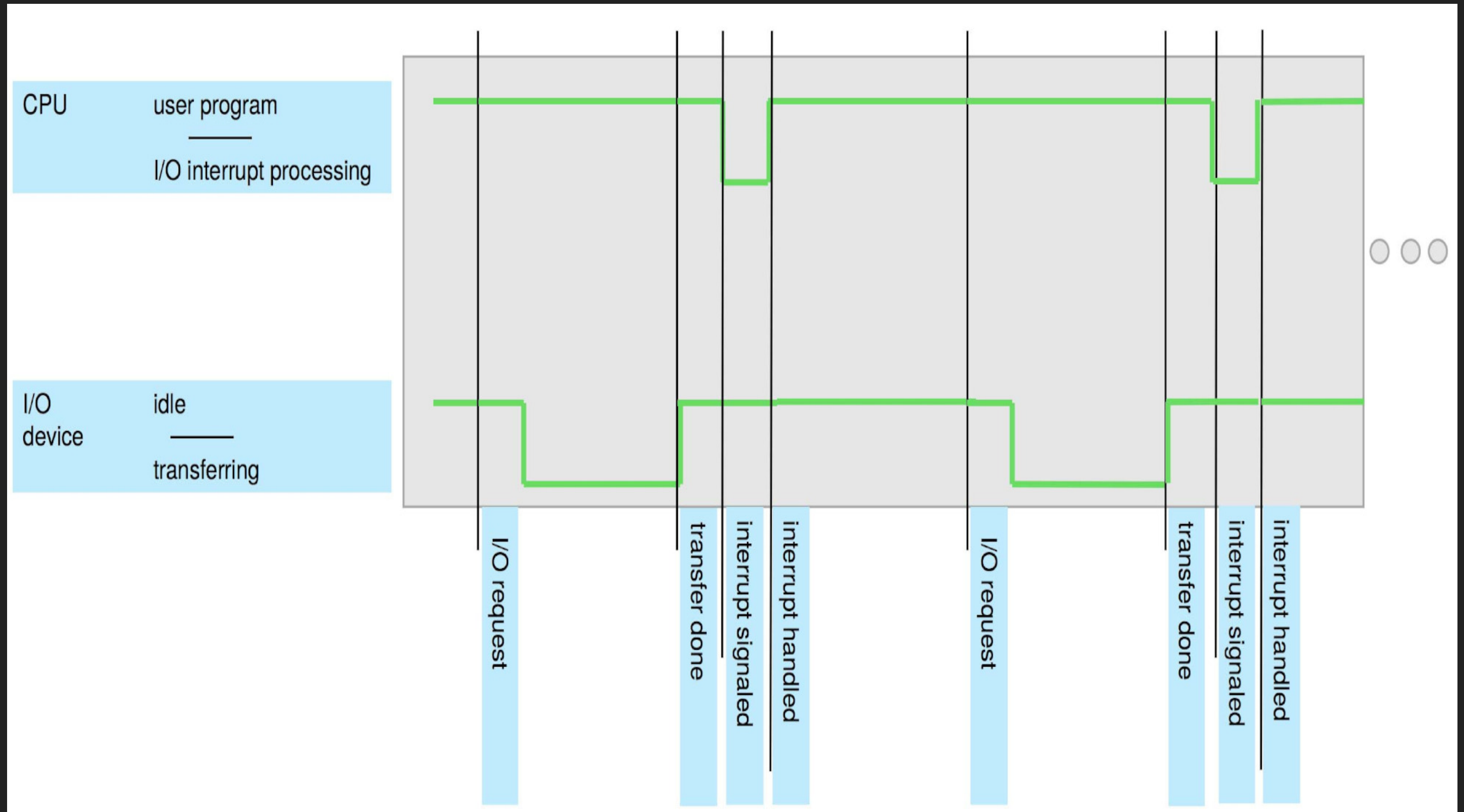
- Ein Interrupt veranlasst die Übertragung der Steuerung an eine sog. Interrupt-Serviceroutine
 - mittels dem sog. Interruptvektor, der die (Speicher-)Adressen aller Serviceroutinen kennt
 - Serviceroutinen sind dann ganz normale Code Abschnitte (Maschinencode) im Speicher
- Eine sog. Trap oder eine Ausnahme, ist ein Software generierter Interrupt, der entweder durch eine Benutzeranfrage oder einen Fehler verursacht wird
- Es existieren also **Hardware-** und **Software-Interrupts** in einem Computersystem

Allgemein gilt: Ein Betriebssystem wird durch Interrupts gesteuert

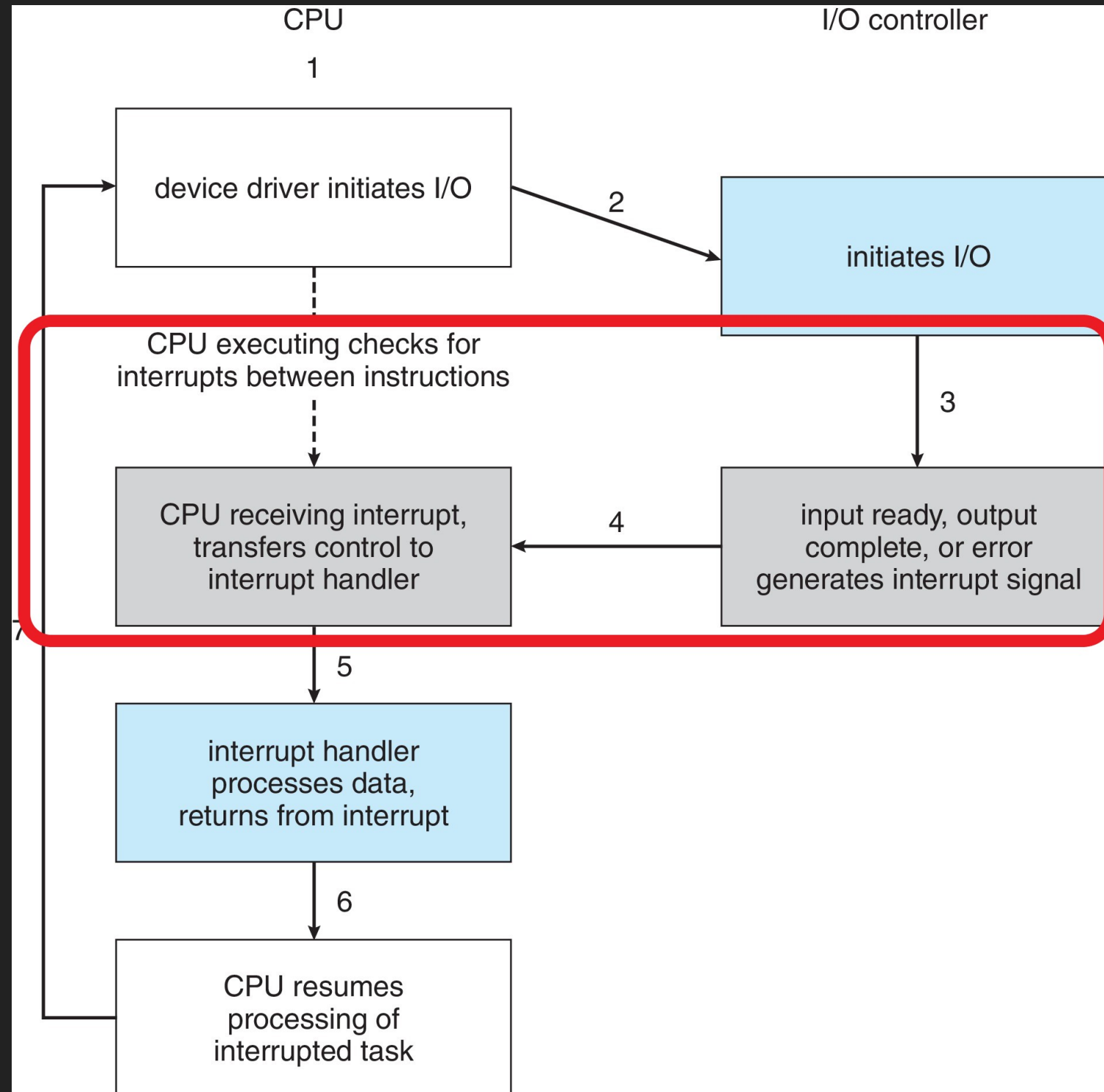
INTERRUPT BEHANDLUNG

- Das Betriebssystem unterbricht das laufende Programm und speichert dessen aktuellen Zustand
 - Hierzu werden u.a. die **CPU-Register** und der **Programmzähler** gespeichert
- Das Betriebssystem springt dann mittels Interruptvektor zur entsprechenden Interrupt-Serviceroutine
 - Separate Codesegmente bestimmen welche konkreten Maßnahmen für den jeweiligen Interrupt ergriffen werden sollen
- Nachdem der Interrupt vollständig bearbeitet wurde, also die Interrupt-Serviceroutine beendet ist:
 - Wiederherstellen der **Register** und
 - Rückkehr zum unterbrochenen Programm via **Programmzähler**

INTERRUPTS VON EINEM I/O GERÄT



INTERRUPT-GESTEUERTE I/O



I/O ARTEN

- **Blocking I/O:** Nach dem Start der I/O Anfrage kehrt die Kontrolle erst nach vollständigem Abschluss der I/O-Operation zum Anwenderprogramm zurück
 - Das Anwendungsprogramm wird schlafen gelegt bis zum Abschluss der I/O Operation
 - Höchstens eine I/O-Anforderung vom Anwendungsprogramm gleichzeitig ausstehend, keine gleichzeitige I/O-Verarbeitung möglich
 - Die CPU kann währenddessen andere Anwendungsprogramme ausführen

Die Verwendung von Blocking I/O wird auch oft als **synchrone Programmierung** bezeichnet

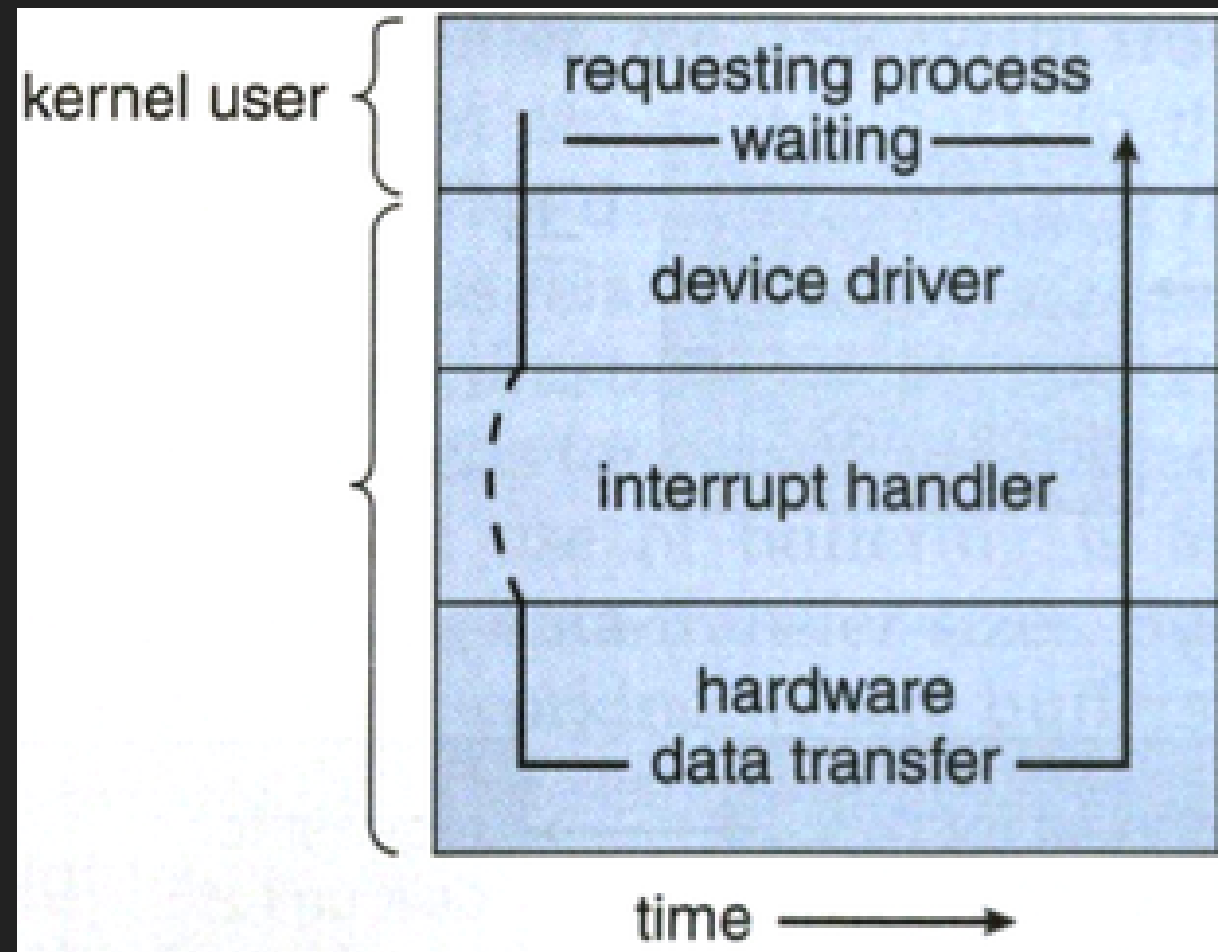
I/O ARTEN

- **Non-Blocking I/O:** Nach dem Start der I/O Anfrage kehrt die Kontrolle direkt zum Anwenderprogramm zurück, ohne auf die Fertigstellung der I/O Operation zu warten
 - Erlaubt parallele Verarbeitung von mehreren I/O-Anforderungen und Verarbeitungen in einem Anwendungsprogramm
 - Das Anwendungsprogramm muss selbstständig prüfen ob die I/O Operation beendet ist (z.B. Prüfung einer gesetzten Variable) oder anderweitig vom Betriebssystem auf die Beendigung aufmerksam gemacht werden:
 - z.B. Signal, Aufruf einer Call-Back Routine (häufig bei GUI's verwendet)

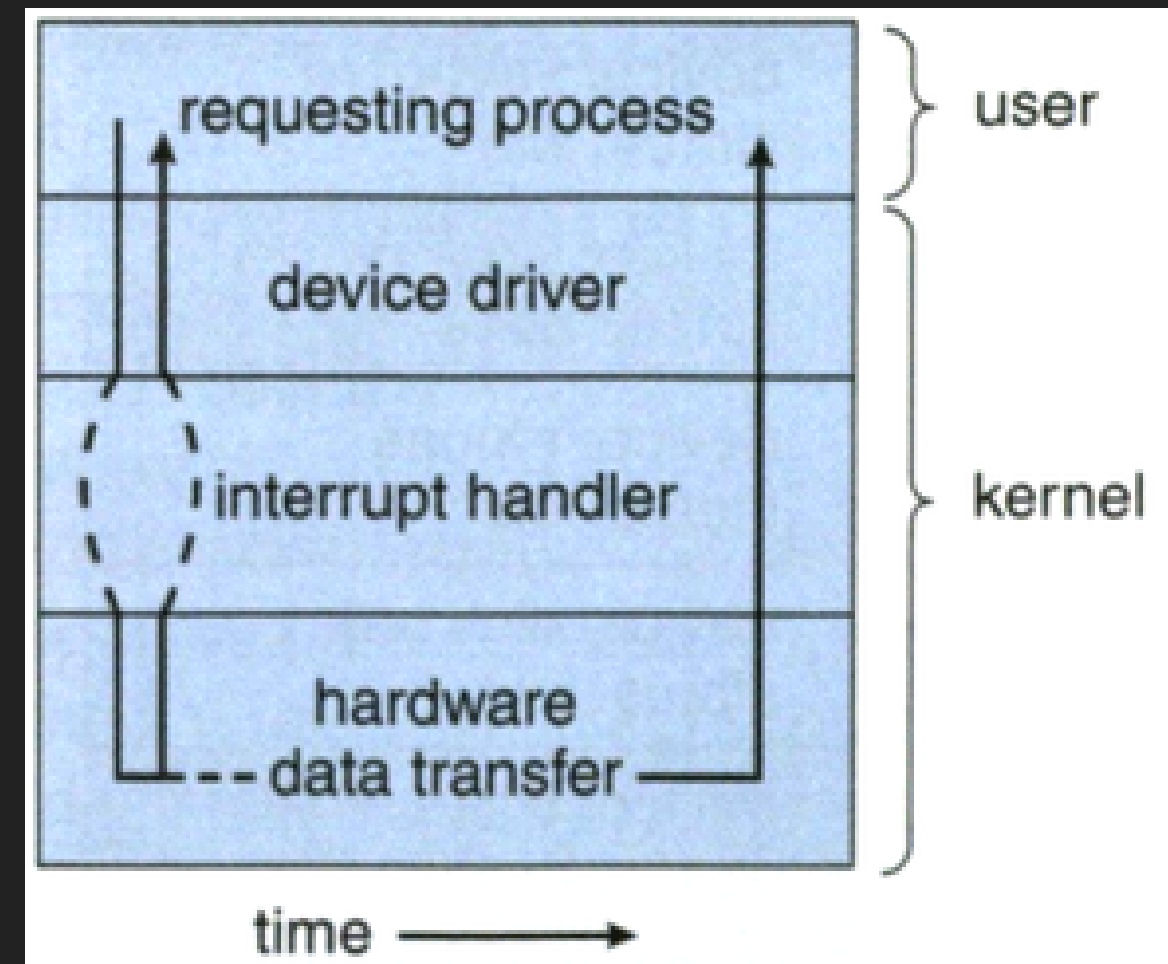
Die Verwendung von Non-Blocking I/O wird auch oft als **asynchrone Programmierung** bezeichnet. Netzwerk- und GUI Programmierung verwenden oft Non-Blocking-I/O

I/O ARTEN

Blockierend / Synchron



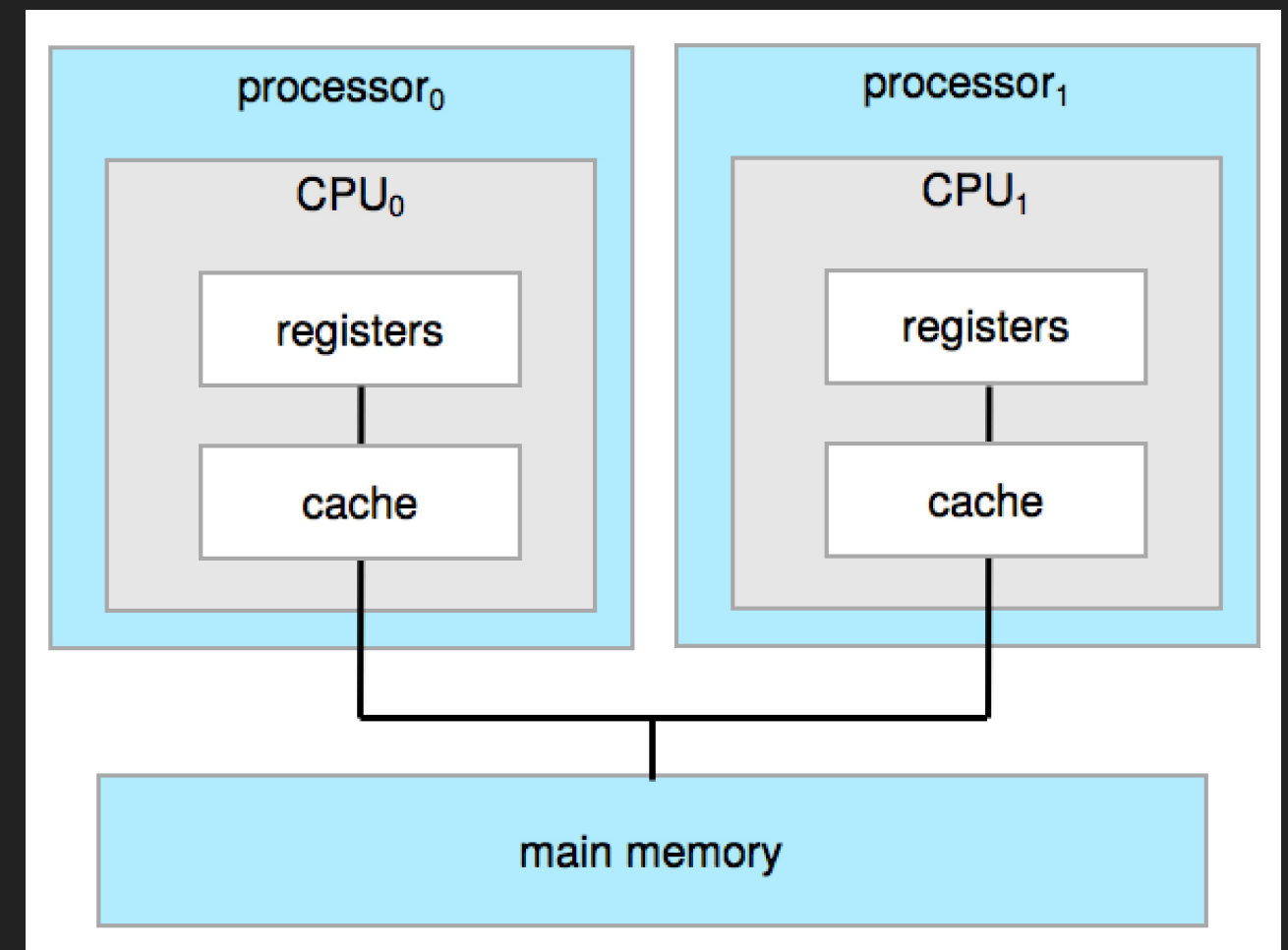
Nicht-Blockierend / Asynchron



COMPUTERSYSTEM ARCHITEKTUR

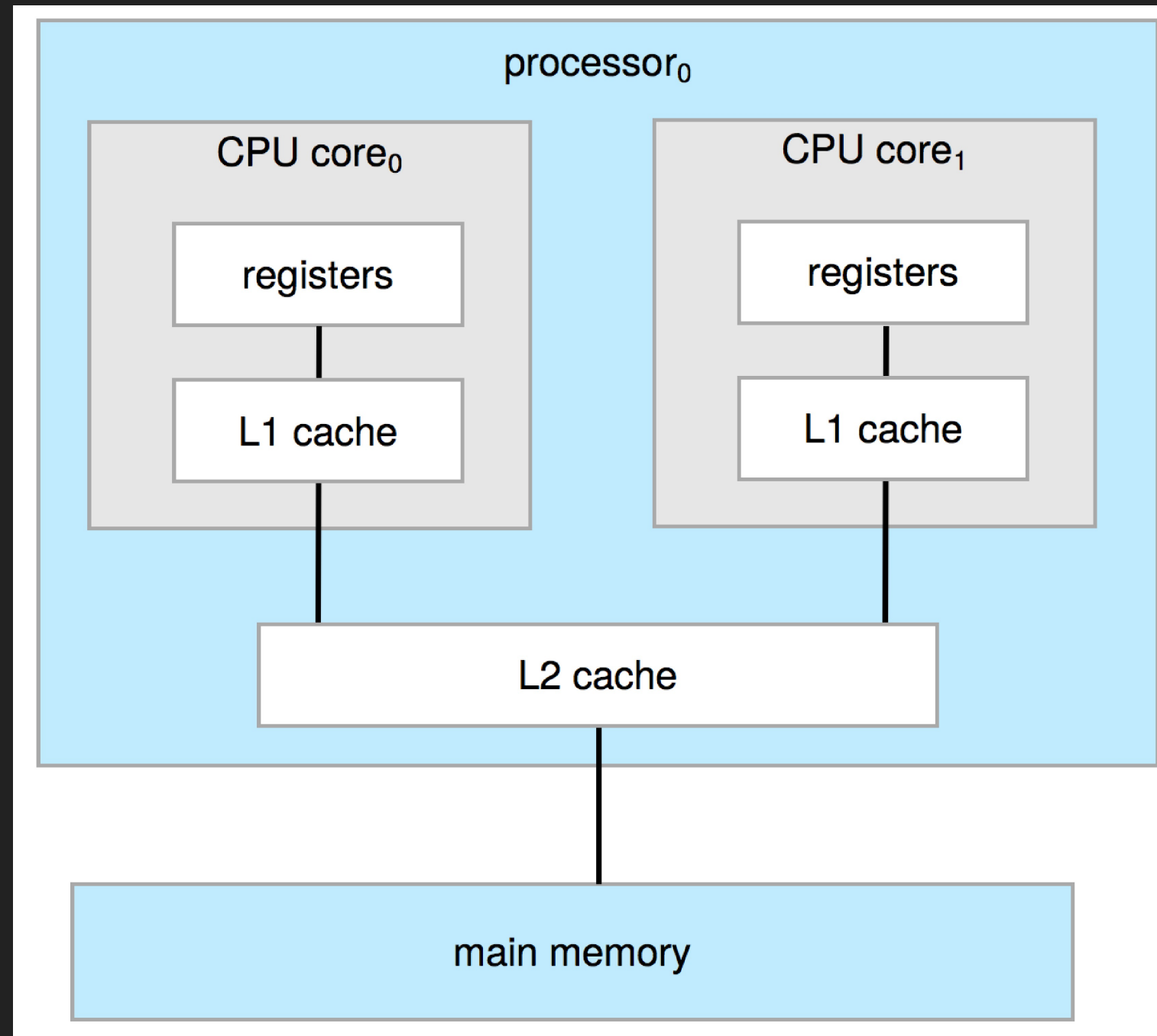
- Früher: Ein einziger Prozessor
- Heute: Überwiegend mehrere Prozessoren (bzw. Prozessorkerne)
 - Zu den Vorteilen gehören:
 - Erhöhter Durchsatz
 - Bessere Skalierung

Symmetrische Multiprocessor Architektur



COMPUTERSYSTEM ARCHITEKTUR

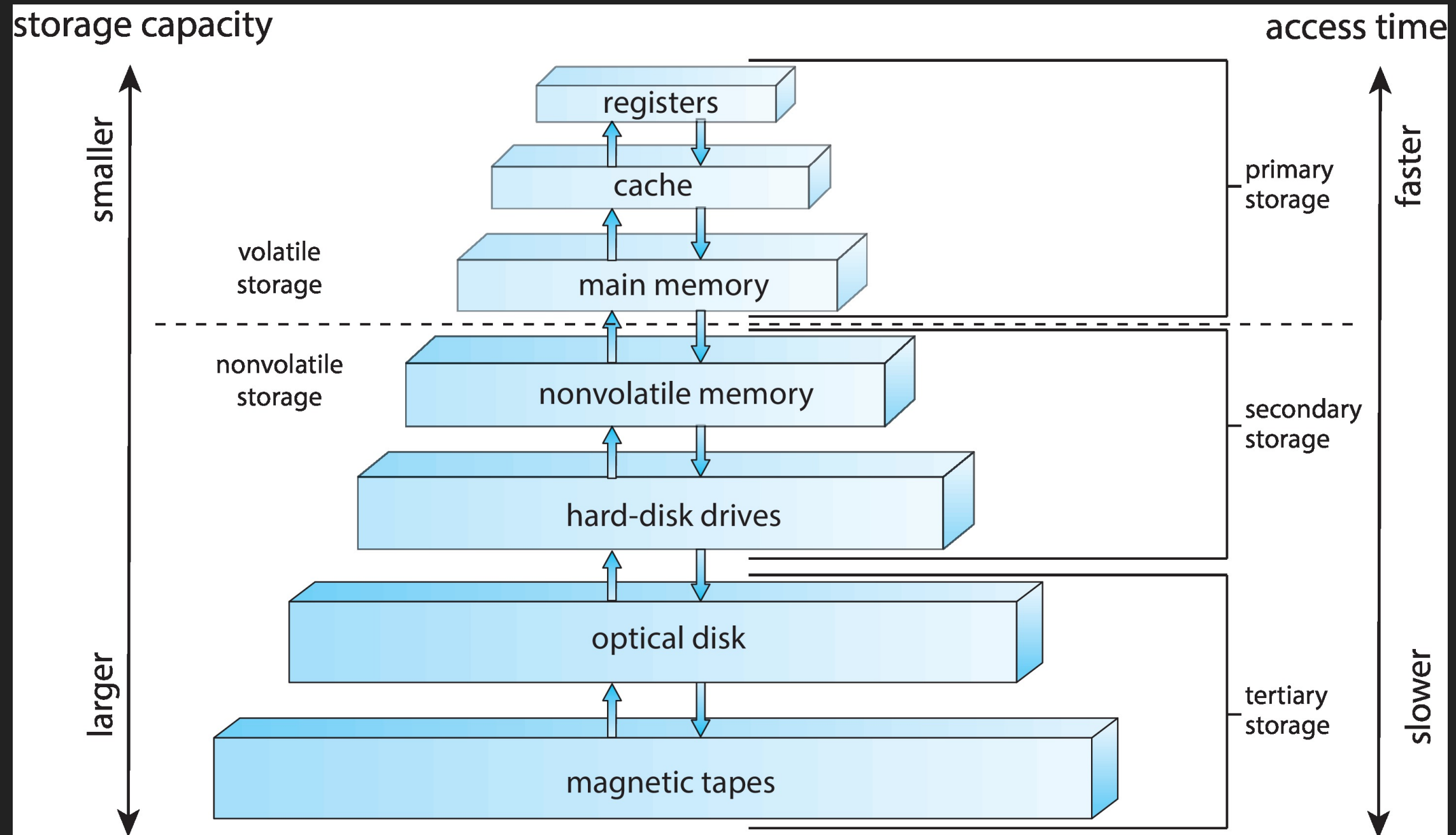
Dual/Multicore Design



SPEICHERARTEN

- **Hauptspeicher (Main Memory)**
 - **Einziges** großes Speichermedium auf das die CPU **direkt** zugreifen kann
 - Dynamic Random-Access Memory (DRAM) - Typischer flüchtiger Speicher mit wahlfreiem/direktem Zugriff (*dynamisch*: muss ständig aufgefrischt (refresh) werden)
- **Sekundärspeicher** - Erweiterung des Hauptspeichers, der eine große nichtflüchtige Speicherkapazität bietet
 - Generelle Bezeichnung: NVM-Geräte (Non-volatile Memory)
 - Festplatten (SSD) - Digitale Speicherbausteine: Flash-Speicher (genauer Flash-EEPROM)
 - Festplatten (HDD) - Metallplatten, die mit magnetischem Aufzeichnungsmaterial beschichtet sind.

ÜBERSICHT SPEICHERGERÄTEHIERACHIE



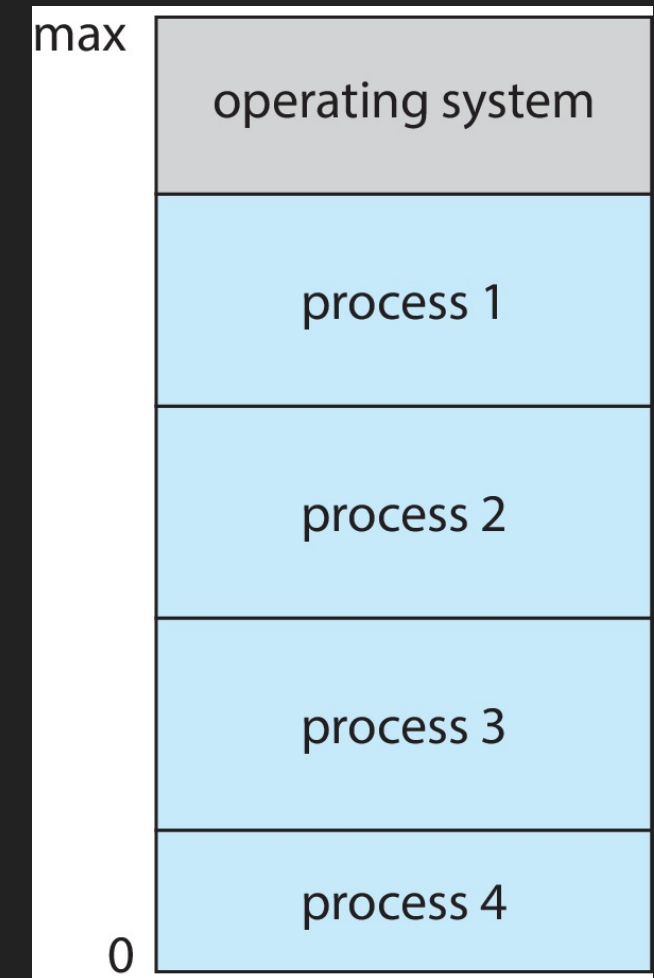
HIERARCHIE VON SPEICHER

- Hierarchisch organisierte Speichersysteme
 - Geschwindigkeit (primär)
 - Kosten
 - Volatilität (Flüchtigkeit)
- **Caching** - Kopieren von Informationen in ein schnelleres Speichersystem
 - der Hauptspeicher kann als Cache für den Sekundärspeicher angesehen werden

ARTEN VON BETRIEBSSYSTEMEN

Früher: Multiprogramming (Batch system)

- **Problem:** Ein Benutzer/Programm kann eine CPU und I/O nicht vollständig auslasten
- **Lösung:** Ein sog. *Batch System* verwaltet viele Programme als sog. Aufgaben(jobs)/Prozesse:
 - Laden **mehrerer** Programme (Code und Daten)
 - Auswahl und Ausführung eines Programms
 - Wenn ein Programm in Ausführung **warten** muss, z.B. auf I/O, wechselt das Betriebssystem zu einem anderen Programm
 - **Zusätzlich:** Wechsel des Programms nach *Zeit* oder bestimmtem *Ressourcenverbrauch*



ARTEN VON BETRIEBSSYSTEMEN

Heute: **Timesharing** (multitasking)

- Logische Erweiterung des Batch Systems
- Typischerweise existieren parallel mehrere **aktive** Aufgaben/Prozesse (tasks)
- CPU wechselt sehr oft zwischen den Prozessen
 - **Ermöglicht interaktives Arbeiten mit einem Computer**
- Primär wird hierbei die Zuteilung von Prozessor(en) zu Prozessen gesteuert und verwaltet

ARTEN VON BETRIEBSSYSTEMEN

Timesharing Multitasking Betrieb



BETRIEBSSYSTEM ABLAUF UND BETRIEB

- Bootstrap-Programm - Code zur Initialisierung des Systems und Laden des Kernels
- Kernel wird geladen und initialisiert das Betriebssystem
- Starten von Systemdiensten (die außerhalb des Kernels bereitgestellt werden)
- Starten von Oberfläche: Shell oder GUI
- Ausführung von Programmen der Benutzer

BETRIEBSSYSTEM ABLAUF UND BETRIEB

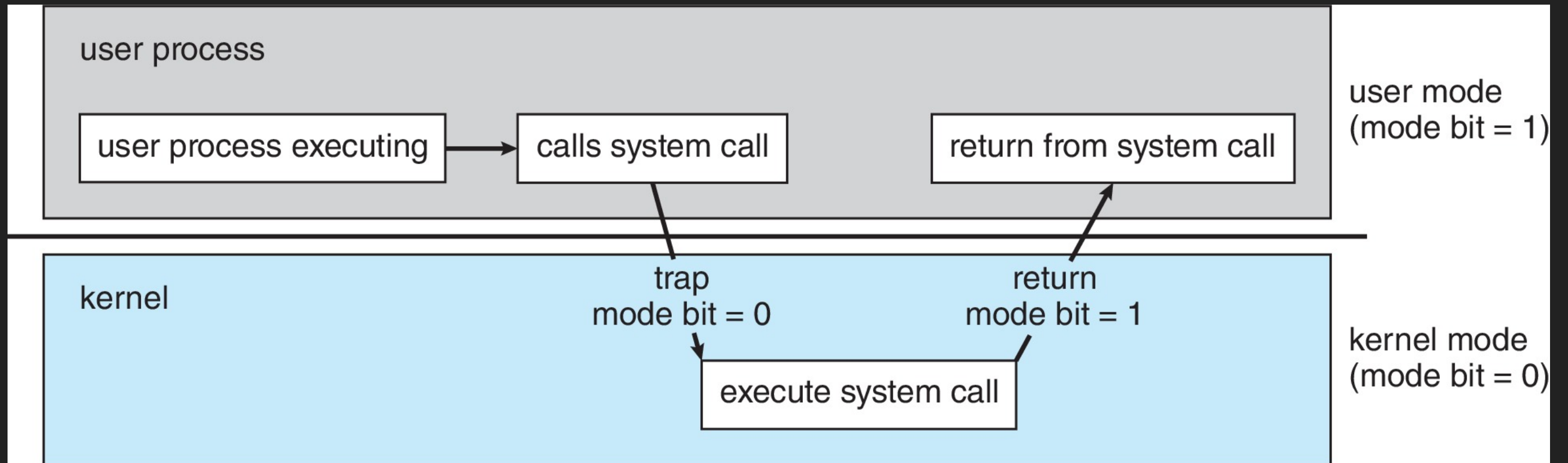
- Wie angemerkt: Der Kernel ist **Interruptgesteuert** (Hard- und Software)
 - **Hardware**-Interrupt durch eines der Geräte
 - **Software**-Interrupt (Ausnahme oder Trap):
 - Softwarefehler (z.B. Division durch Null) behandeln
 - Aufruf eines Betriebssystemservices - **Systemaufruf** (System Call)
 - Andere Prozess-relevante Probleme behandeln: z.B. Endlosschleifen, Synchronisation, ...

BETRIEBSMODI

Der sog Dual-Modus-Betrieb ermöglicht es dem Betriebssystem, sich selbst und andere Systemkomponenten zu schützen

- Benutzermodus (**user mode**) und Kernelmodus (**kernel mode**)
- Die Hardware (CPU) stellt dafür ein Modusbit bereit
 - Unterscheidung ob Benutzercode oder Kernelcode ausgeführt wird
 - *Privilegierte Anweisungen sind nur im Kernelmodus ausführbar*
- Systemaufrufe (system calls) wechseln vom Benutzermodus (**user mode**) in den Kernelmodus (**kernel mode**)
 - Modusbit wird automatisch nach Beendigung des Aufrufs zurück auf Benutzermodus (**user mode**) gesetzt

SYSTEM CALLS



PROZESSMANAGEMENT

- Ein **Prozess** ist ein Programm, das sich in **Ausführung** befindet
 - ... *oder dazu bereit umgehend ausgeführt zu werden*
- Ein Prozess ist somit eine **aktive** Einheit
- Ein Programm ist eine **passive** Einheit

PROZESSMANAGEMENT

- Ein Prozess benötigt **Ressourcen**, um seine Aufgabe zu erfüllen:
 - CPU, Speicher, I/O, Dateien
 - Alle wiederverwendbaren Ressourcen (z.B. Speicher) werden nach Beendigung des Prozesses zurückgegeben
- **Single-Thread-Prozesse** haben einen Programmzähler (*Program Counter [PC]*), der die Position der auszuführenden Anweisung angibt
 - Der Prozessor führt alle Anweisungen strikt nacheinander aus
- **Multi-Thread-Prozesse** haben einen Programmzähler **pro Thread**
- **Time Sharing**: Gleichzeitigkeit entsteht durch Multiplexen der CPUs zwischen den Prozessen/Threads (Scheduling)

AUFGABEN BETRIEBSSYSTEM PROZESSMANAGEMENT

Das Betriebssystem ist für die folgenden Aktivitäten beim Prozessmanagement verantwortlich:

- **Erzeugen** und **Entfernen** von Prozessen
- **Anhalten** und **Fortsetzen** von Prozessen
- Bereitstellung von Mechanismen für die
 - Prozesskommunikation
 - Prozesssynchronisation und
 - ggf. Deadlock-Verarbeitung

SPEICHERMANAGEMENT

- Um Anweisungen eines Programmes auszuführen zu können, müssen sich **alle** auszuführenden Anweisungen im **Hauptspeicher** befinden
- *Auch die Daten, die vom Programm aktuell verarbeitet werden, müssen sich ebenfalls im Hauptspeicher befinden*



Nur Anweisungen die gerade unmittelbar in Ausführung sind und die Daten die gerade verarbeitet werden, müssen zwingend im Hauptspeicher vorhanden sein.

D.h. dass Anweisungen und Daten nicht zu jeder Zeit vollständig im Hauptspeicher sein müssen!

Sie werden automatisch bei Bedarf durch das Betriebssystem nachgeladen.

SPEICHERMANAGEMENT

- Die Speicherverwaltung bestimmt, was und zu welchem Zeitpunkt sich etwas im Hauptspeicher befindet
 - Optimierung der CPU-Auslastung und der Reaktionszeit für Benutzer
- Speicherverwaltungstätigkeiten
 - Verfolgen, welche Teile des Hauptspeichers derzeit verwendet werden und von wem
 - Entscheiden, welche Prozesse (oder Teile davon) und Daten in den und aus dem Hauptspeicher bewegt werden sollen
 - Zuweisen und Freigeben von Speicherplatz nach Bedarf (`mmap`, `malloc`, `free`)
 - Zugriffskontrolle: Wer darf wie (`rw`~~`x`~~) und auf was im Hauptspeicher zugreifen

DATEISYSTEMMANAGEMENT

- Ein Betriebssystem bietet eine einheitliche, logische Sicht auf Informationen
 - Die Speichereinheit ist eine **Datei**, welche von einem sog. **Dateisystem** bereitgestellt wird
 - Dateien werden typischerweise in sog. **Verzeichnissen** verwaltet, die in der Regel eine hierarchische Struktur aufweisen
- Ein konkretes Dateisystem selbst beschreibt und implementiert, wie Informationen (logisch) angeordnet sein müssen, damit diese als Datei bereitgestellt werden kann
- Zur Verwaltung stellt das Betriebssystem Funktionen bereit um mit den Dateien zu arbeiten
- Weiterhin wichtig **Zugriffskontrolle**: *Wer darf auf welche/s Datei/Verzeichnis wie zugreifen*

DATEISYSTEMMANAGEMENT

Zu den vom Betriebssystem bereitgestellten Funktionen gehören u.a.:

- **Erstellen** und **Löschen** von Dateien und Verzeichnissen
- Funktionen zur **Manipulation** von Dateien und Verzeichnissen
- Speichern von Daten im Hauptspeicher in Dateien auf nichtflüchtige Speichermedien
- Mapping von Dateien zwischen Hauptspeicher und Sekundärspeicher
 - Und dessen Synchronisation

MASSENSPEICHERVERWALTUNG

Normalerweise werden Festplatten verwendet, um Daten zu speichern, die nicht in den Hauptspeicher passen, oder Daten, die für einen *längeren* Zeitraum aufbewahrt werden müssen.



Die gesamte **Geschwindigkeit** des Computersystems hängt essentiell vom Festplattensubsystem und seinen Algorithmen ab

- Betriebssystem Aktivitäten u.a.:
 - Mount und unmount
 - Allokation von Speicher
 - Verwaltung des freien Speichers
 - Partitionierung
- Einige Speicher müssen nicht schnell sein: Solche Tertiärspeicher sind z.B. optische Speicher, Magnetband, etc.

CACHING

Caching ist ein wichtiges Prinzip, das auf vielen Ebenen in einem Computer durchgeführt wird (Hardware, Betriebssystem, Software)



Grundprinzip: Benötigte Daten werden von einem langsameren Speicher in einem schnelleren Speicher kopiert und vom schnelleren Speicher bereitgestellt

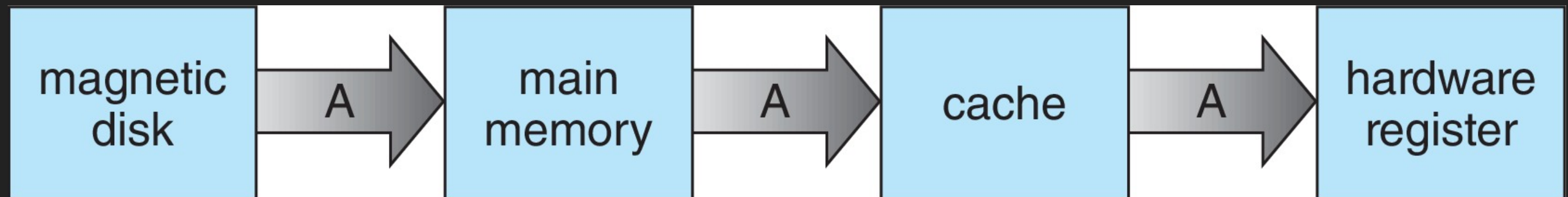
- Schnellerer Speicher (*der Cache*) wird zuerst überprüft, um festzustellen, ob die Daten vorhanden sind
 - Wenn ja, werden die Daten direkt aus dem Cache verwendet (schnell)
 - Wenn nicht, werden die Daten zuerst in den Cache kopiert und anschließend von dort verwendet
- Cache ist typischerweise kleiner als der kaschierte Speicher
 - Cache-Management ist ein wichtiges Designproblem und sehr komplex
 - Insbesondere: Wahl von Cache-Größe und Ersetzungsstrategie für den Cache

SPEICHERTYPEN UND ZUGRIFFSZEITEN

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

DATEN VON DISK ZUM REGISTER

- Die **Multitaskingumgebungen** (Betriebssystem) muss darauf achten, den gültigen (aktuellsten) Wert zu verwenden → (Disk und Hauptspeicher)



- Die **Multiprozessorumgebung** (CPU(s)) muss die Cache-Kohärenz in der Hardware (CPU-Caches) so gewährleisten, dass alle CPUs den gültigen (aktuellsten) Wert in ihrem Cache haben und mit diesem Wert rechnen

I/O SUBSYSTEM

Ein Zweck des Betriebssystems ist es, Besonderheiten von Hardwaregeräten vor dem Benutzer zu verbergen (**abstrakter Rechner**)

Zuständig ist hierfür das **I/O-Subsystem**

- Speicherverwaltung von I/O einschließlich Pufferung (temporäres Speichern von Daten während der Übertragung),
- Caching (Speichern von Daten(-teilen) in schnellerem Speicher zur Leistungssteigerung)
- Spooling (Verwaltung von Aufträgen im Stapelverfahren, z.B. Druckerwarteschlange)
- Bereitstellung einer einheitlichen Geräte-Treiber-Schnittstelle

SCHUTZ UND SICHERHEIT

- **Schutz:** Mechanismen zur Steuerung des Zugriffs von Prozessen oder Benutzern auf die vom Betriebssystem definierten Ressourcen
- **Sicherheit:** Härtung des Systems gegen interne und externe Angriffe
 - Verschiedenste implizite und explizite Maßnahmen verfügbar
- *Grundsätzlich unterscheiden heutige Systeme zwischen Benutzern:*
 - Benutzeridentitäten (Benutzer-IDs, Sicherheits-IDs) identifizieren einen Benutzer des Systems — Ein Nutzer muss eindeutig identifizierbar sein!
 - Die Benutzer-ID (userid), kann allen Dateien und Prozessen dieses Benutzers zugeordnet werden — auf dieser Basis kann dann eine Zugriffskontrolle umgesetzt werden
 - Eine Gruppen-ID (groupid) ermöglicht es mehrere Benutzer in einer Gruppe zusammenzufassen und zu verwalten und so eine Zugriffskontrolle umzusetzen, die mehrere Benutzer (eine bestimmte Gruppe) betrifft

AUFBAU UND STRUKTUR VON BETRIEBSSYSTEMEN

BETRIEBSSYSTEM DIENSTE

Betriebssysteme bieten eine Umgebung für die Ausführung von Programmen und Diensten für Programme und Benutzer Eine Reihe von Betriebssystemdiensten stellt Funktionen zur Verfügung, die für den Benutzer hilfreich sind:

- **Benutzeroberfläche** - Fast alle Betriebssysteme haben eine Benutzeroberfläche (UI). Variiert zwischen:
 - **Kommandozeile (CLI), Grafischer Benutzeroberfläche (GUI)**
- **Programmausführung** - Das System muss in der Lage sein, ein Programm:
 - in den Speicher zu laden, auszuführen und die Ausführung entweder **normal** oder **anormal** (Fehler/Exception) zu beenden
- **I/O-Operationen** - Ein laufendes Programm kann I/O erfordern, was eine Datei oder ein I/O-Gerät beinhalten kann

BETRIEBSSYSTEM DIENSTE: BENUTZER

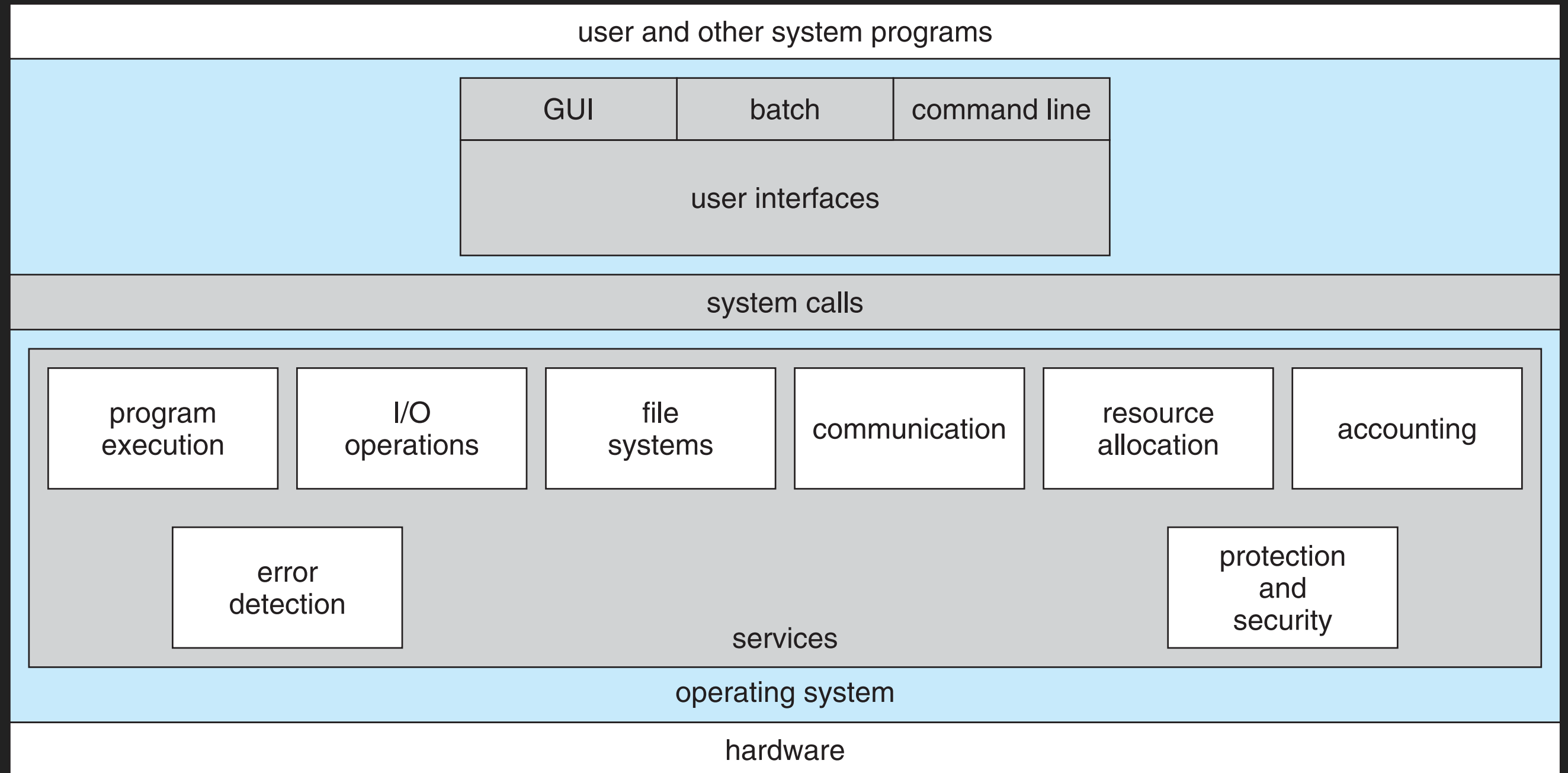
- **Verwendung des Dateisystems** - Programme müssen Dateien und Verzeichnisse lesen und schreiben, erstellen und löschen, durchsuchen, Dateiinformationen anzeigen und Berechtigungen verwalten
- **Kommunikation** - Prozesse müssen Informationen austauschen auf demselben Computer oder in einem Netzwerk.
 - Erfolgt über einen gemeinsamen Speicher oder durch Nachrichtenübermittlung
- **Fehlererkennung** - Betriebssystem muss ständig auf mögliche Fehler achten:
 - CPU und Speicherhardware, I/O-Geräte, Anwenderprogramm
 - *Für jede Art von Fehler muss das Betriebssystem die entsprechenden Maßnahmen ergreifen, um eine korrekte und konsistente Datenverarbeitung zu gewährleisten.*
 - Debugging-Funktionen helfen Benutzern und Programmierern Fehler zu erkennen und zu beheben.

BETRIEBSSYSTEM DIENSTE: EFFIZIENTER BETRIEB

Betriebssystemfunktionen zum effizienten Betrieb des Systems

- **Ressourcenzuweisung** - In Mehrbenutzersystemen oder wenn mehrere Programme gleichzeitig ausgeführt werden:
 - Arten: CPU-Zyklen, Hauptspeicher, Dateispeicher, I/O-Geräte.
- **Accounting** - Wer nutzt wie viel und welche Arten von Ressourcen
- **Schutz und Sicherheit** - Eigentümer von Informationen in einem Mehrbenutzersystem wollen den Zugriff auf ihre Informationen kontrollieren; gleichzeitig ausgeführte Prozesse dürfen sich nicht gegenseitig stören
 - Der Schutz beinhaltet die Sicherstellung, dass der gesamte Zugriff auf die Systemressourcen kontrolliert wird
 - Die Sicherheit des Systems vor externen Zugriff erfordert eine Benutzerauthentifizierung, die bis hin zum Schutz externer I/O-Geräte vor ungültigen Zugriffsversuchen reicht

ÜBERSICHT VON BETRIEBSSYSTEM DIENSTEN



BETRIEBSSYSTEM BENUTZERINTERFACES

Command Line Interfaces (CLI)

- CLI oder Command Interpreter ermöglichen eine direkte Eingabe von Befehlen:
 - Manchmal im Kernel implementiert, manchmal von Systemprogramm.
 - Heute überwiegend als Shells bereitgestellt: BASH, ZSH, SH, ...
- Aufgaben: Holt primär einen Befehl vom Benutzer und führt ihn aus.
 - Grundsätzlich: Aufruf und Steuerung von Programmen
 - Oft sind auch Befehle direkt in der Shell eingebaut (built-ins):
 - Schleifen, Conditionals, Historie, Kill, ...

BENUTZEROBERFLÄCHEN: LINUX CLI

BASH Shell in URXVT (Terminalemulator) und TMUX

The screenshot shows a Linux terminal window with a dark theme. The top bar indicates the window title is "machinar:0.3.2 vim machinar". The terminal is divided into two main sections. The left section displays system status and a process list. The right section shows a C program named "helper.c" being edited in vim.

System Status:

```
1  [|||] 2.6% Tasks: 194, 934 thr; 1 running
2  [|||] 4.6% Load average: 0.73 1.11 1.23
3  [|||] 6.0% Uptime: 02:17:25
4  [|||] 3.3%
Mem[|||||] 4.62G/31.2G
Swp[ ] 0K/23.3G
```

Process List:

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
3300	ar	20	0	2320M	235M	73024	S	4.6	0.7	7:58.25	/usr/bin/gnome-shell
5455	ar	20	0	840M	136M	80796	S	3.3	0.4	3:42.26	/opt/google/chrome-beta/chrome -
3660	ar	20	0	1048M	82632	58312	S	2.6	0.3	2:48.70	gnome-control-center network sho
12943	ar	20	0	840M	136M	80796	S	2.0	0.4	1:08.30	/opt/google/chrome-beta/chrome -
3198	ar	20	0	455M	92964	56436	S	1.3	0.3	3:54.51	/usr/lib/xorg/Xorg vt2 -displayf
12947	ar	20	0	840M	136M	80796	S	1.3	0.4	1:08.70	/opt/google/chrome-beta/chrome -
12942	ar	20	0	840M	136M	80796	S	1.3	0.4	0:46.52	/opt/google/chrome-beta/chrome -
3065	ar	20	0	27096	4768	3120	R	0.7	0.0	0:00.97	htop
9039	ar	20	0	30408	4180	2752	S	0.7	0.0	0:22.38	tmux
4128	ar	20	0	1678M	367M	154M	S	0.7	1.2	5:26.35	/opt/google/chrome-beta/chrome
15936	ar	20	0	1554M	332M	119M	S	0.7	1.0	0:50.52	/usr/lib/libreoffice/program/sof
4666	ar	20	0	701M	91632	65620	S	0.7	0.3	0:07.49	/opt/google/chrome-beta/chrome -
2463	root	20	0	817M	42800	27660	S	0.7	0.1	0:01.50	/usr/bin/dockerd -H fd://
2806	root	20	0	817M	42800	27660	S	0.7	0.1	0:01.61	/usr/bin/dockerd -H fd://
886	root	20	0	35652	1880	1648	S	0.7	0.0	0:00.28	/usr/sbin/irqbalance --foreground

File System:

```
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit
drwxr-xr-x 19 root root 3340 Aug 7 13:38 dev
drwxr-xr-x 160 root root 12288 Aug 7 15:49 etc
drwxr-xr-x 5 root root 4096 Jun 17 10:29 home
lrwxrwxrwx 1 root root 29 May 25 06:22 initrd.img -> boot/initrd.img-4.9.0-8-amd64
lrwxrwxrwx 1 root root 36 May 25 06:22 initrd.img.old -> boot/initrd.img-4.18.0-0.bpo.1-am
d64
drwxr-xr-x 21 root root 4096 Jun 12 11:02 lib
drwxr-xr-x 2 root root 4096 Dec 10 2018 lib64
drwx----- 2 root root 16384 Jun 19 2017 lost+found
drwxr-xr-x 5 root root 4096 Sep 13 2018 media
drwxr-xr-x 2 root root 4096 Jun 19 2017 mnt
drwxr-xr-x 5 root root 4096 May 20 13:00 opt
dr-xr-xr-x 291 root root 0 Aug 7 13:37 proc
drwx----- 16 root root 4096 Jul 17 10:14 root
drwxr-xr-x 31 root root 1100 Aug 7 15:13 run
drwxr-xr-x 2 root root 12288 Jun 12 11:02 sbin
drwxr-xr-x 2 root root 4096 Jun 19 2017 srv
dr-xr-xr-x 13 root root 0 Aug 7 15:53 sys
drwxrwxrwt 16 root root 24576 Aug 7 15:53 tmp
drwxr-xr-x 12 root root 4096 Jun 26 2018 usr
drwxr-xr-x 12 root root 4096 Jun 20 2017 var
lrwxrwxrwx 1 root root 26 May 25 06:22 vmlinuz -> boot/vmlinuz-4.9.0-8-amd64
lrwxrwxrwx 1 root root 33 May 25 06:22 vmlinuz.old -> boot/vmlinuz-4.18.0-0.bpo.1-amd64
```

C Program (helper.c):

```
r/T/V/B/c/s/helper.c buffers
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5
6 void hexDump(char *desc, void *addr, int len, int offset) {
7     int i;
8     unsigned char buff[16];
9     unsigned char *pc = (unsigned char*)addr;
10
11     // Output description if given.
12     if (desc != NULL)
13         printf ("%s:\n", desc);
14
15     // Process every byte in the data.
16     for (i = 0; i < len; i++) {
17         // Multiple of 16 means new line (with line offset).
18
19         if ((i % 16) == 0) {
20             // Just don't print ASCII for the zeroth line.
21             if (i != 0)
22                 printf(" %s\n", buff);
23
24             // Output the offset.
25             printf("0x%lx  %04x ", (long unsigned int) ((char *)addr+i), i);
26         }
27
28         // Now the hex code for the specific character.
29         printf(" %02x", pc[i+offset]);
30
31         // And store a printable ASCII character for later.
32         if ((pc[i+offset] < 0x20) || (pc[i+offset] > 0x7e)) {
33             buff[i % 16] = '.';
34         } else {
35             buff[i % 16] = pc[i+offset];
36         }
37
38         buff[(i % 16) + 1] = '\0';
39     }
40
41     // Pad out last line if not exactly 16 characters.
42     while ((i % 16) != 0) {
43         printf(" ");
44         i++;
45     }
46
47     // And print the final ASCII bit.
```

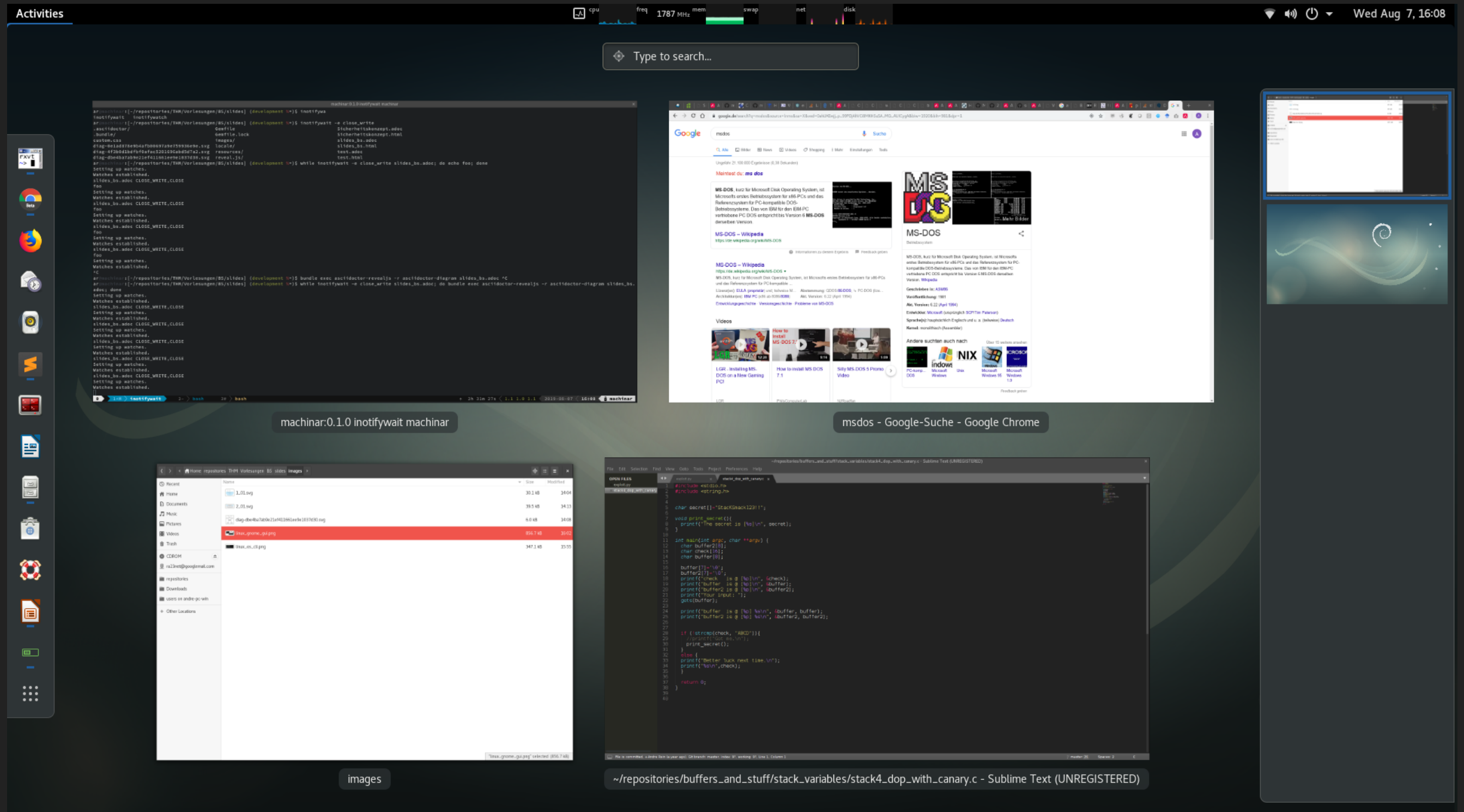
The terminal window also shows a status bar at the bottom with information about the current session, including the window title, the current directory, and the current file being edited.

BETRIEBSSYSTEM BENUTZERINTERFACES (GUI)

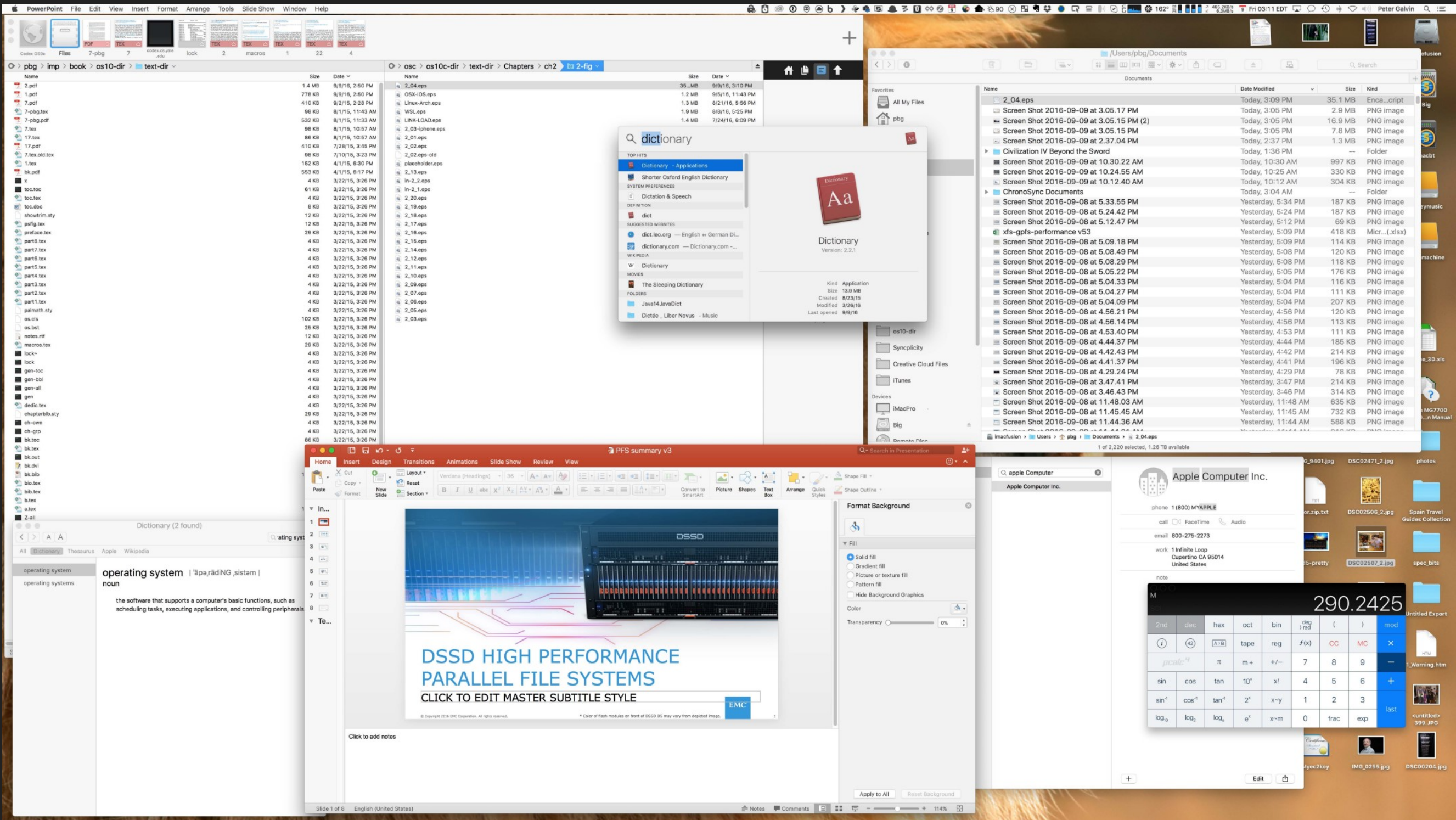
Graphical User Interface (GUI)

- Benutzerfreundliche Desktopoberfläche mit Symbolen
 - Bedienung: Normalerweise Maus, Tastatur und Monitor
 - Symbole stellen Dateien, Programme, Aktionen usw. dar.
 - Verschiedene Mausklicks über Objekte in der Oberfläche führen zu verschiedenen Aktionen (Informationen, Optionen, Ausführungsfunktion).
 - Erfunden bei Xerox PARC
- *Viele Systeme verfügen heute sowohl über CLI- als auch über GUI-Schnittstellen.*
 - Microsoft Windows ist eine GUI mit CLI "command"-Shell.
 - Apple Mac OS X ist eine "Aqua" GUI-Schnittstelle mit UNIX-Kernel darunter und verfügbaren Shells.
 - Unix und Linux haben CLI mit optionalen GUI-Schnittstellen (KDE, GNOME).

BENUTZEROBERFLÄCHEN: LINUX GUI



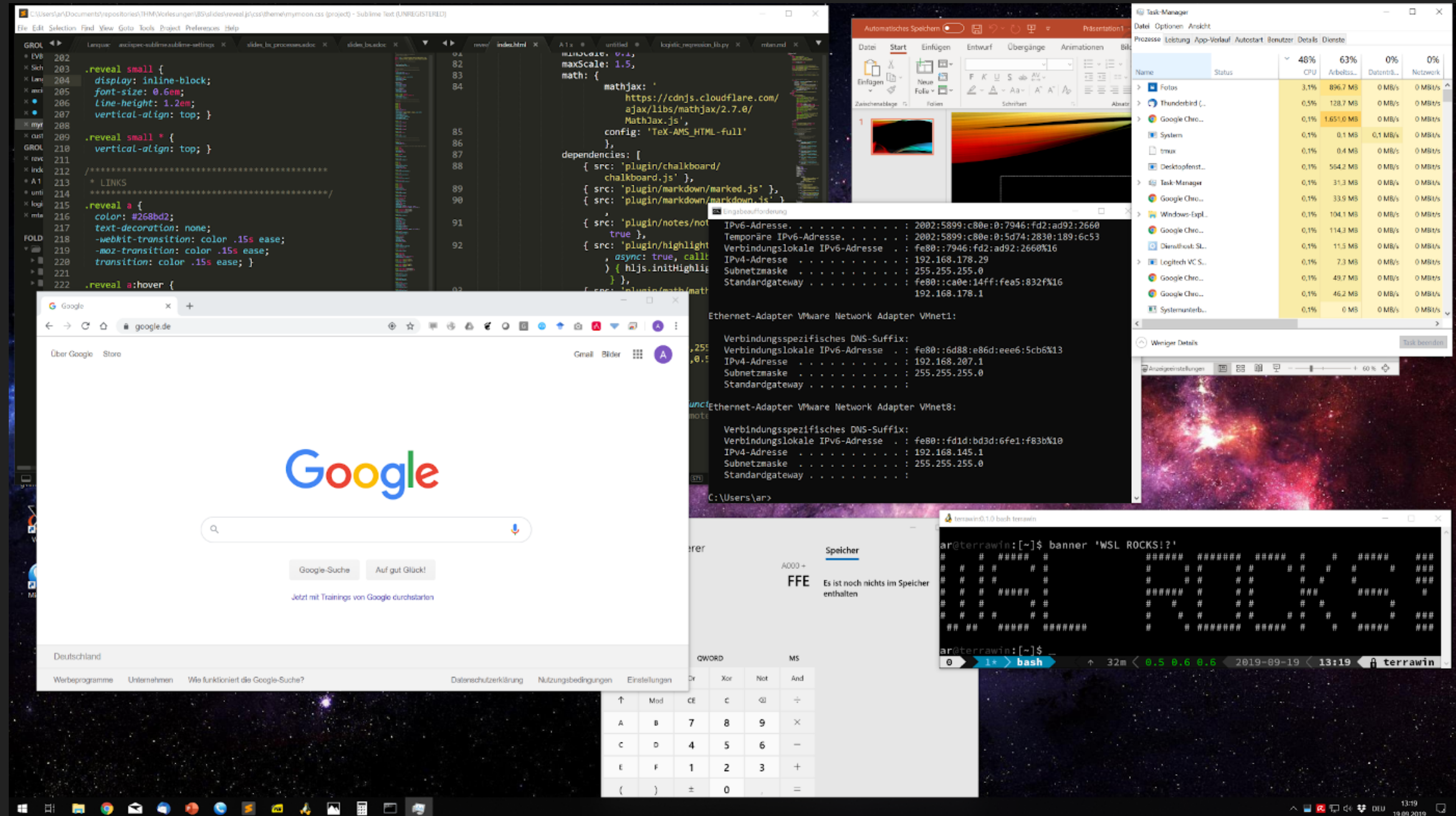
BENUTZEROBERFLÄCHEN: MACOS



BENUTZEROBERFLÄCHEN: IOS



BENUTZEROBERFLÄCHEN: WINDOWS 10



BENUTZEROBERFLÄCHEN

- Was denken Sie ist für das Betriebssystem aufwändiger zu verwalten? — Ein CLI wie z.B. Bash oder — Ein Desktopsystem
- Begründen Sie Ihre Vermutung!

SYSTEMAUFRUFE (SYSTEM CALLS)



AUFPASSEN!



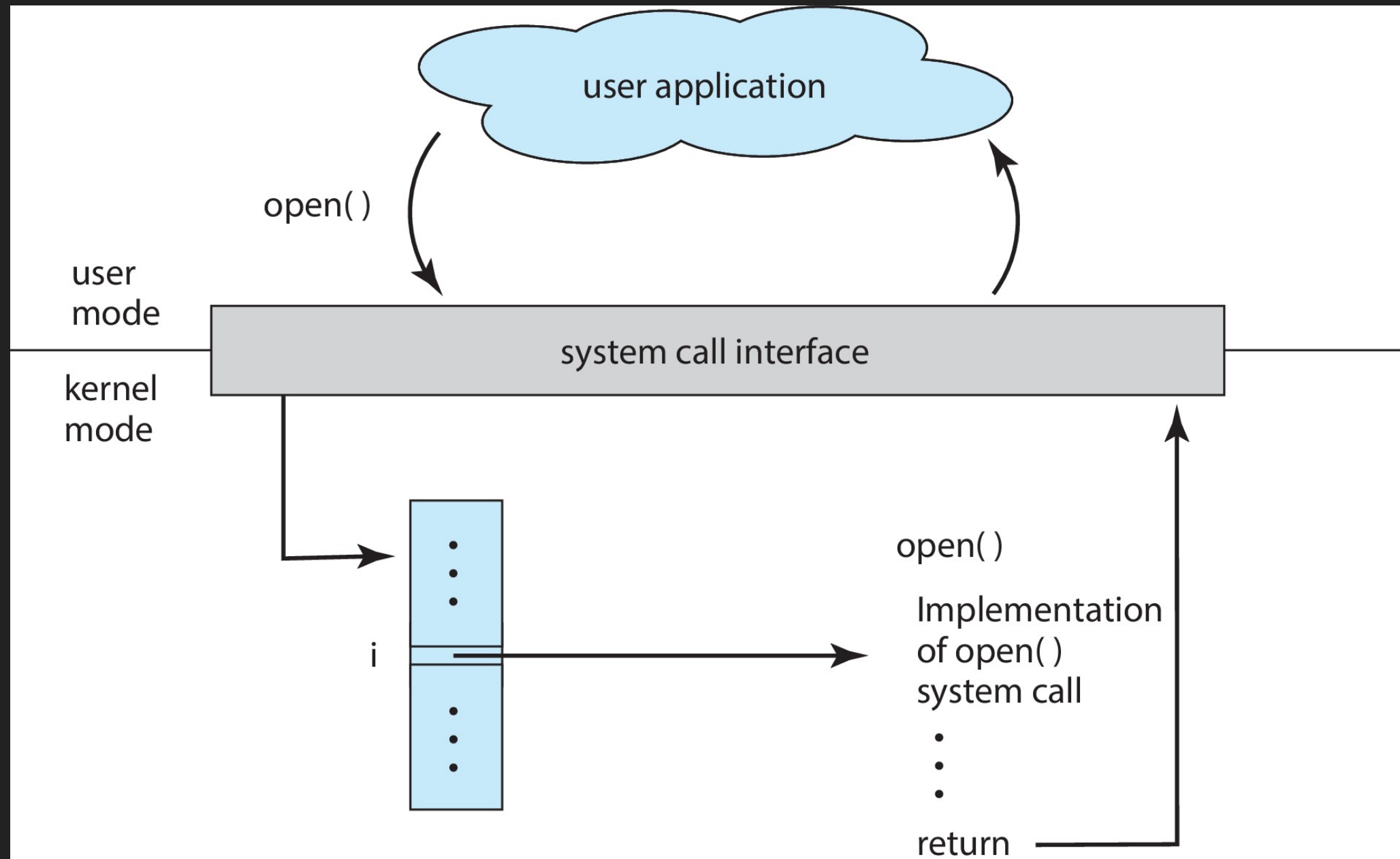
SYSTEMAUFRUFE (SYSTEM CALLS)

Systemaufrufe (nachfolgend *system calls* genannt) sind **Softwareinterrupts**, die es Benutzerprogrammen erlauben vom Betriebssystem bereitgestellte Funktionen im privilegierten **Kernelmodus** aufzurufen.

- Bereitgestellt vom Kernel
- Zugriff üblicherweise über eine API (Application Programming Interface)
 - Win32 API für Windows
 - POSIX API Posix basierte Systeme
 - Nahezu alle Unix-derivate, Linux und Max OS X)
- Typischerweise implementiert in einer systemnahen Hochsprache (z.B. C oder C++)

SYSTEM CALLS API

Beispiel: Öffnen einer Datei mittels der Funktion `open()`.



SYSTEM CALLS (BEISPIEL)

FRAGE: WAS MACHT FOLGENDES CODEFRAGMENT? 

```
1 int main(void) {
2     register int a    asm("rax") = 1;
3     register int b    asm("rdi") = 1;
4     register char* c   asm("rsi") = "ABC\n";
5     register int d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```

SYSTEM CALLS (BEISPIEL)

```
1 int main(void) {
2     register int    a    asm("rax") = 1;
3     register int    b    asm("rdi") = 1;
4     register char*  c    asm("rsi") = "ABC\n";
5     register int    d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```

Terminal: compile and run

```
$gcc -o syscall_write_asm syscall_write_asm.c
$./syscall_write_asm
ABC
```

Terminal: strace (system-call trace)

```
$strace ./syscall_write_asm
...
write(1, "ABC\n", 4)
...
```

SYSTEM CALLS (BEISPIEL)

```
1 #include <stdio.h>
2 int main(void) {
3     printf("ABC\n");
4     return 0;
5 }
```

```
1 int main(void) {
2     register int    a    asm("rax") = 1;
3     register int    b    asm("rdi") = 1;
4     register char*  c    asm("rsi") = "ABC\n";
5     register int    d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```

SYSTEM CALLS (BEISPIEL)

```
1 #include <stdio.h>
2 int main(void) {
3     printf("ABC\n");
4     return 0;
5 }
```

```
1 #include <unistd.h>
2 int main(void) {
3     write(1, "ABC\n", 4);
4     return 0;
5 }
```

```
1 #include <unistd.h>
2 #include <sys/syscall.h>
3 int main(void) {
4     syscall(SYS_write, 1, "ABC\n", 4);
5     return 0;
6 }
```

```
1 int main(void) {
2     register int a    asm("rax") = 1;
3     register int b    asm("rdi") = 1;
4     register char* c   asm("rsi") = "ABC\n";
5     register int d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```


SYSTEM CALLS (BEISPIEL)

Source Code 1 (libc syscall)

```
1 #include <unistd.h>
2 #include <sys/syscall.h>
3 int main(void) {
4     syscall(SYS_write, 1, "ABC\n", 4);
5     return 0;
6 }
```

Source Code (assembler)

```
1 int main(void) {
2     register int    a    asm("rax") = 1;
3     register int    b    asm("rdi") = 1;
4     register char*  c    asm("rsi") = "ABC\n";
5     register int    d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```

Disassembly (libc syscall)

```
1 4004f6 <main>:
2 4004f6: push    %rbp
3 4004f7: mov     %rsp,%rbp
4 4004fa: mov     $0x4,%ecx
5 4004ff: mov     $0x4005a4,%edx
6 400504: mov     $0x1,%esi
7 400509: mov     $0x1,%edi
8 40050e: mov     $0x0,%eax
9 400513: callq   4003f0 <syscall@plt>
10 400518: mov     $0x0,%eax
11 40051d: pop     %rbp
12 40051e: retq
```

Disassembly (assembler)

```
1 4004a6 <main>:
2 4004a6: push    %rbp
3 4004a7: mov     %rsp,%rbp
4 4004aa: mov     $0x1,%eax
5 4004af: mov     $0x1,%edi
6 4004b4: mov     $0x400554,%esi
7 4004b9: mov     $0x4,%edx
8 4004be: syscall
9 4004c0: mov     $0x0,%eax
10 4004c5: pop     %rbp
11 4004c6: retq
```


SYSTEM CALLS (BEISPIEL)

Disassembly libc
syscall Version -
Bereitet die
Register für den
Funktionsaufruf
vor.

```
1 4004f6 <main>:  
2 4004f6: push    %rbp  
3 4004f7: mov     %rsp,%rbp  
4 4004fa: mov     $0x4,%ecx  
5 4004ff: mov     $0x4005a4,%edx  
6 400504: mov     $0x1,%esi  
7 400509: mov     $0x1,%edi  
8 40050e: mov     $0x0,%eax  
9 400513: callq   4003f0 <syscall@plt>  
10 400518: mov     $0x0,%eax  
11 40051d: pop     %rbp  
12 40051e: retq
```

Source Code Glibc
für syscall
Funktion

```
1 /* Usage: long syscall (syscall_number, arg1, arg2, arg3, arg4, arg5, arg6)  
2    We need to do some arg shifting, the syscall_number will be in rax. */  
3 .text  
4 ENTRY (syscall)  
5     movq %rdi, %rax /* Syscall number -> rax. */  
6     movq %rsi, %rdi /* shift arg1 - arg5. */  
7     movq %rdx, %rsi  
8     movq %rcx, %rdx  
9     movq %r8, %r10  
10    movq %r9, %r8  
11    movq 8(%rsp),%r9 /* arg6 is on the stack. */  
12    syscall /* Do the system call. */  
13    cmpq $-4095, %rax /* Check %rax for error. */  
14    jae SYSCALL_ERROR_LABEL /* Jump to error handler if error. */  
15    ret /* Return to caller. */  
16 PSEUDO_END (syscall)
```

source: https://github.com/bminor/glibc/blob/master/sysdeps/unix/sysv/linux/x86_64/syscall.S

SYSTEM CALLS (BEISPIEL)

Das Ziel der verschachtelten Funktionsaufrufe ist es, die Register so zu belegen, sodass die `syscall`-Instruktion korrekt aufgerufen wird. (So wie es das allererste Codebeispiel getan hat.)

register	libc	syscall	assembler
rcx (ecx)			
rdx (edx)			0x4
rsi (esi)			ABC\n
rdi (edi)			0x1
rax (eax)			0x1

```
1 int main(void) {
2     register int    a    asm("rax") = 1;
3     register int    b    asm("rdi") = 1;
4     register char*  c    asm("rsi") = "ABC\n";
5     register int    d    asm("rdx") = 4;
6     asm("syscall");
7     return 0;
8 }
```

```
1 4004a6 <main>:
2 4004a6: push    %rbp
3 4004a7: mov     %rsp,%rbp
4 4004aa: mov     $0x1,%eax
5 4004af: mov     $0x1,%edi
6 4004b4: mov     $0x400554,%esi
7 4004b9: mov     $0x4,%edx
8 4004be: syscall
9 4004c0: mov     $0x0,%eax
10 4004c5: pop     %rbp
11 4004c6: retq
```

SYSTEM CALLS (BEISPIEL)

Die Variante in C mit dem Aufruf `syscall(SYS_write, 1, "ABC\n", 4);` gibt die Registereingangsbelegung an

register	libc	syscall	assembler
rcx (ecx)	0x4		
rdx (edx)	0x4005a4		0x4
rsi (esi)	0x1		ABC\n
rdi (edi)	0x1		0x1
rax (eax)	0x0		0x1

```
1 4004f6 <main>:
2 4004f6: push    %rbp
3 4004f7: mov     %rsp,%rbp
4 4004fa: mov     $0x4,%ecx
5 4004ff: mov     $0x4005a4,%edx
6 400504: mov     $0x1,%esi
7 400509: mov     $0x1,%edi
8 40050e: mov     $0x0,%eax
9 400513: callq   4003f0 <syscall@plt>
10 400518: mov     $0x0,%eax
11 40051d: pop     %rbp
12 40051e: retq
```

SYSTEM CALLS (BEISPIEL)

Nun folgt die Anpassung der Register in der internen `syscall`-Funktion der Glibc.

register	libc	syscall	assembler
rcx (ecx)	0x4		
rdx (edx)	0x4005a4	0x4	0x4
rsi (esi)	0x1	0x4005a4	ABC\n
rdi (edi)	0x1	0x1	0x1
rax (eax)	0x0	0x1	0x1

```
1 ENTRY (syscall)
2  movq %rdi, %rax          /* Syscall number -> rax.*/
3  movq %rsi, %rdi          /* shift arg1 - arg5.*/
4  movq %rdx, %rsi
5  movq %rcx, %rdx
6  movq %r8, %r10
7  movq %r9, %r8
8  movq 8(%rsp),%r9         /* arg6 is on the stack.*/
9  syscall                  /* Do the system call.*/
10 cmpq $-4095, %rax        /* Check %rax for error.*/
11 jae SYSCALL_ERROR_LABEL /* poss. jump error handler.*/
12 ret                      /* Return to caller. */
13 PSEUDO_END (syscall)
```

SYSTEM CALLS (BEISPIEL)

Der Aufruf der `syscall` Funktion der Glibc `syscall(SYS_write, 1, "ABC\n", 4)`; ergibt dann das gleiche Ergebnis wie im ursprüngliche Assemblercode angegeben.

register	libc	syscall	assembler
rcx (ecx)	0x4		
rdx (edx)	0x4005a4	0x4	0x4
rsi (esi)	0x1	0x4005a4	ABC\n
rdi (edi)	0x1	0x1	0x1
rax (eax)	0x0	0x1	0x1

```
1 4004a6 <main>:  
2 4004a6: push    %rbp  
3 4004a7: mov     %rsp,%rbp  
4 4004aa: mov     $0x1,%eax  
5 4004af: mov     $0x1,%edi  
6 4004b4: mov     $0x400554,%esi  
7 4004b9: mov     $0x4,%edx  
8 4004be: syscall  
9 4004c0: mov     $0x0,%eax  
10 4004c5: pop     %rbp  
11 4004c6: retq
```



Jede angegebene Variante des Programms ruft letztendlich `syscall` auf und "macht" somit das exakt gleiche.

SYSTEM CALLS



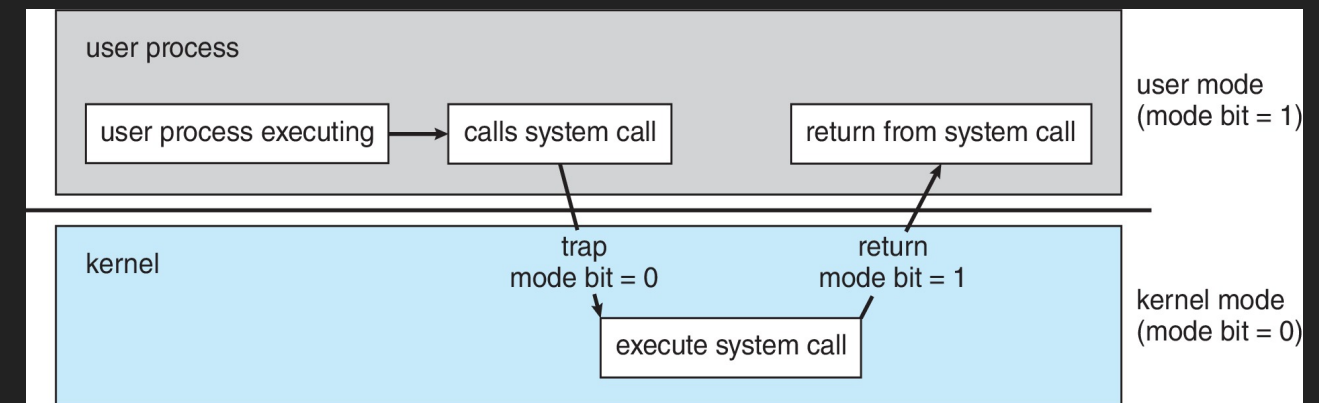
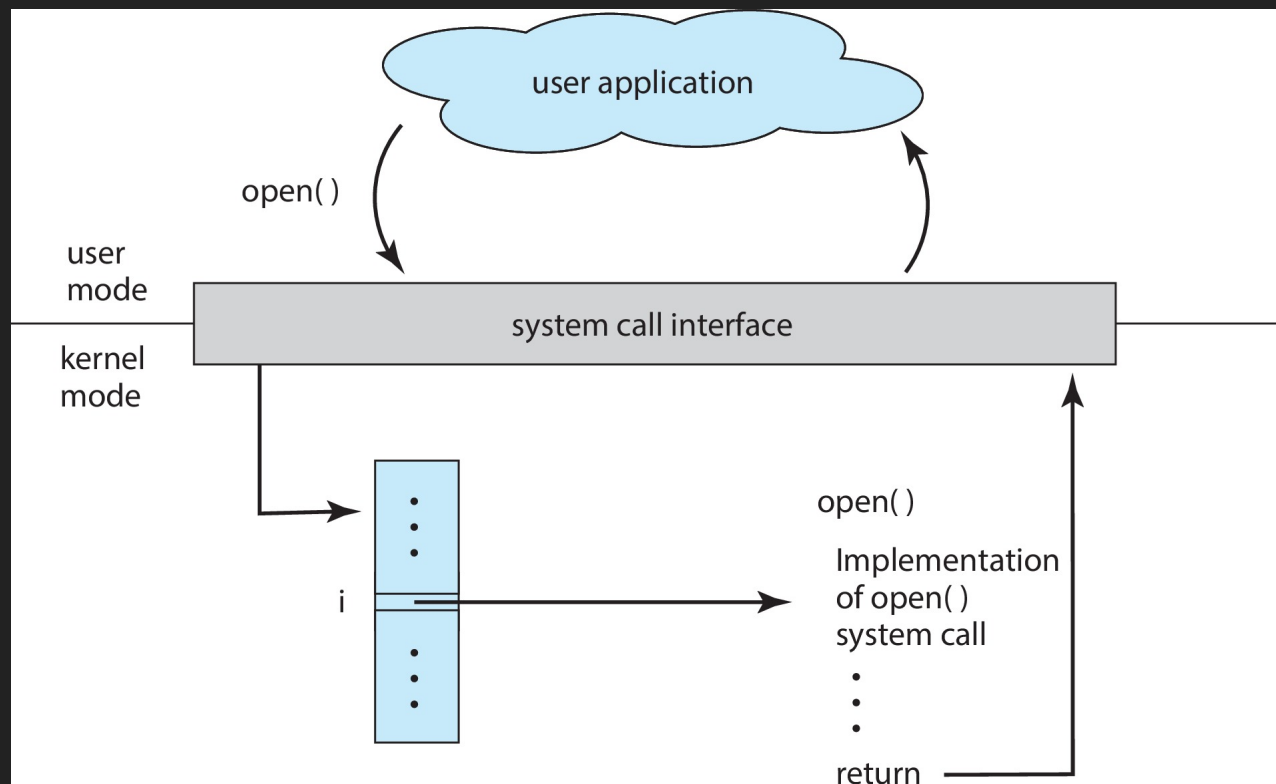
Die `syscall` Funktion wird für **jeden** system call aufgerufen, nicht nur für `write`

```
1 /* Usage: long syscall (syscall_number, arg1, arg2, arg3, arg4, arg5, arg6)
2    We need to do some arg shifting, the syscall_number will be in rax. */
3 .text
4 ENTRY (syscall)
5     movq %rdi, %rax /* Syscall number -> rax. */
6     movq %rsi, %rdi /* shift arg1 - arg5. */
7     movq %rdx, %rsi
8     movq %rcx, %rdx
9     movq %r8, %r10
10    movq %r9, %r8
11    movq 8(%rsp), %r9 /* arg6 is on the stack. */
12    syscall /* Do the system call. */
13    cmpq $-4095, %rax /* Check %rax for error. */
14    jae SYSCALL_ERROR_LABEL /* Jump to error handler if error. */
15    ret /* Return to caller. */
16 PSEUDO_END (syscall)
```

Wenn alle Parameter korrekt in Registern und auf dem Stack platziert sind, springt die **Instruktion** `syscall` in den Kernel.

SYSTEM CALLS

- Jeder system call hat eine festgelegt Nummer und wird mittels der `syscall`-Instruktion aufgerufen (*Im Beispiel war die Nummer für `write=1`*)
- Das *System Call Interface* ruft den angeforderten system call im Betriebssystem auf (*System Call Table*) und gibt entsprechend der Spezifikation einen Rückgabewert zurück
 - Wechsel user-mode → kernel-mode → user-mode



SYSTEM CALLS (PARAMETERÜBERGABE)

- Verschiedene Methoden (kombinierbar):
 - Parameter in Registern
 - Parameter auf dem Stack
 - Anwendung `push()` → Betriebssystem `pop()`
 - Parameter in anderem Speicherbereich (also nicht Stack)
 - Hier wird nur die Adresse zum jeweiligen Speicher via Register übergeben
 - Struktur und Zugriff der Daten ist dann ein Implementierungsdetail



Wie wir gesehen haben verwendet Linux z.B. eine Mischform von Registern und Stack.

SYSTEM CALLS (FRAGE)

Die Verwendung des system calls `write` war sehr einfach. Warum macht es für einen Programmierer aber Sinn `printf()` oder `puts()` für Ausgaben zu verwenden, anstatt eine der angegebenen Varianten?

- Portabilität
 - Kompiliert und läuft auf jedem System das die API unterstützt
- Implizite Fehlerbehandlung
- Einfachheit durch das Verstecken von Details
 - Einige system calls sind nicht so einfach zu handhaben wie `write`

SYSTEM CALLS (TYPEN)

Typ	Beispiele	Typ	Beispiele
Prozesskontrolle	• Prozess erzeugen, terminieren, beenden, abbrechen, laden, ausführen • warten, signalisieren • Speicherallokation • Debugging	Dateimanagement	• Datei erzeugen, löschen, öffnen, schließen • lesen, schreiben • Dateiattribute setzen und holen
Gerätemanagement	• Gerät anfordern, freigeben • lesen, schreiben • Gerätattribute setzen und holen • logisches einbinden und auswerfen	Information Zeit, Attribute, ...	• Zeit/Datum holen oder setzen • Systeminformationen holen oder setzen • get and set process, file, or device attributes

SYSTEM CALLS (TYPEN)

Typ	Beispiele	Typ	Beispiele
Kommunikation	• Kommunikationsverbindung erzeugen, löschen • Nachrichten senden und empfangen • Shared-memory Model: ■ Speicherregionen erzeugen und Zugriff erhalten	Sicherheit und Schutz	• Zugriffskontrolle auf Ressourcen • Holen und setzen von Berechtigungen • Zugriffe erlauben oder ablehnen

SYSTEM CALLS IN WINDOWS UND LINUX/UNIX

	Windows	Unix/Linux
Prozesskontrolle	• <code>CreateProcess()</code> • <code>ExitProcess()</code> • <code>WaitForSingleObject()</code>	• <code>fork()</code> • <code>exit()</code> • <code>wait()</code>
Dateimanagement	• <code>CreateFile()</code> • <code>ReadFile()</code> • <code>WriteFile()</code> • <code>CloseHandle()</code>	• <code>open()</code> • <code>read()</code> • <code>write()</code> • <code>close()</code>
Gerätemanagement	• <code>SetConsoleMode()</code> • <code>ReadConsole()</code> • <code>WriteConsole()</code>	• <code>ioctl()</code> • <code>read()</code> • <code>write()</code>

SYSTEM CALLS IN WINDOWS UND LINUX/UNIX

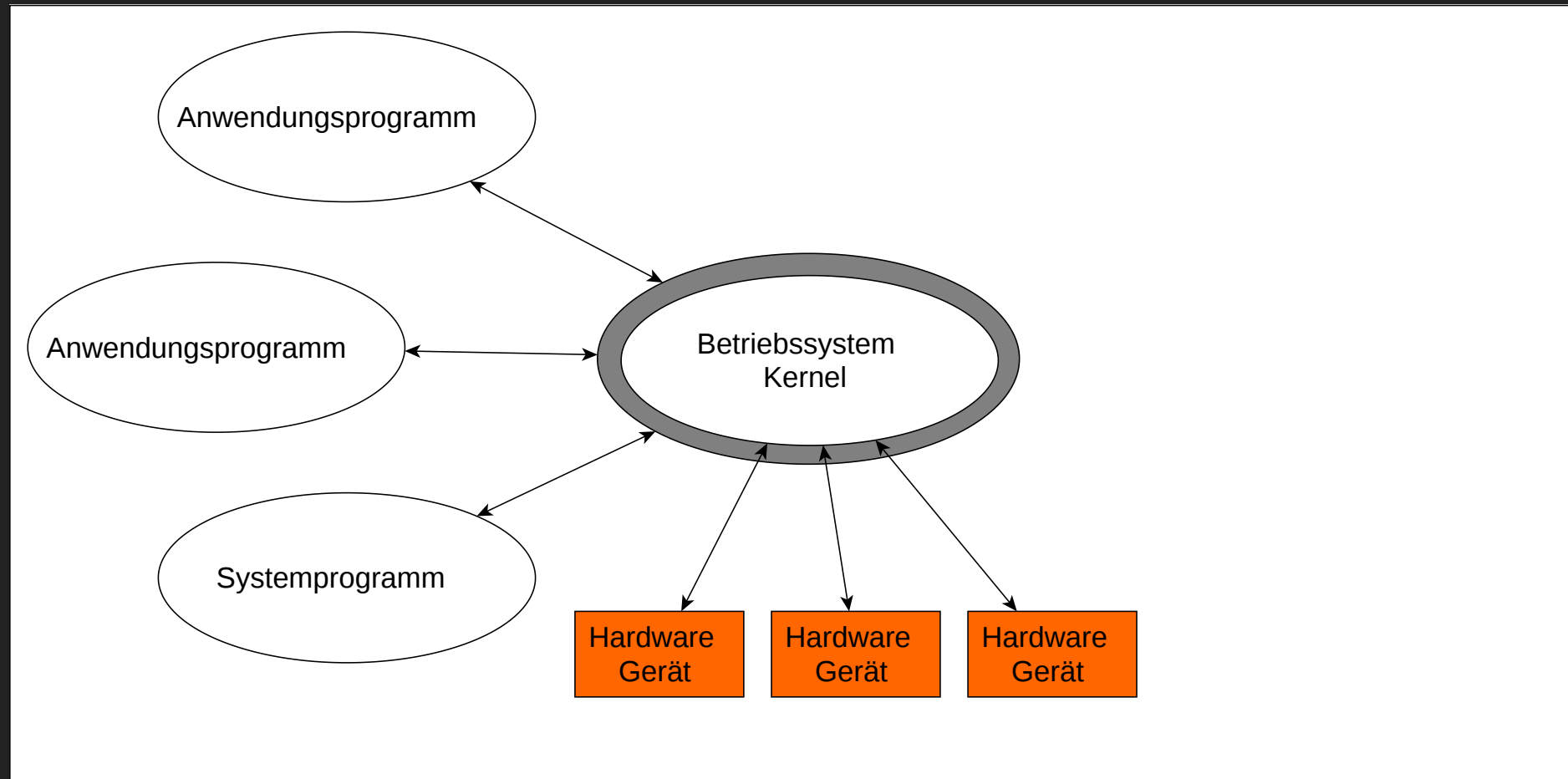
	Windows	Unix/Linux
Information/Betrieb	• <code>GetCurrentProcessId()</code> • <code>SetTimer()</code> • <code>Sleep()</code>	• <code>getpid()</code> • <code>alarm()</code> • <code>sleep()</code>
Kommunikation	• <code>CreatePipe()</code> • <code>CreateFileMapping()</code> • <code>MapViewOfFile()</code>	• <code>pipe()</code> • <code>shm_open()</code> • <code>mmap()</code>
Sicherheit	• <code>SetFileSecurity()</code> • <code>InitializeSecurityDescriptor()</code> • <code>SetSecurityDescriptorGroup()</code>	• <code>chmod()</code> • <code>umask()</code> • <code>chown()</code>

SYSTEM CALLS (FURTHER READING)

- [system call table linux x64](#)
- [Kernel Code des `write` system calls](#)
- [Kernel Documentation: system calls](#)
- [Kernel Documentation: Adding a new system call](#)
- [Linux system call entry point](#)
- [Linux system call Dispatcher](#)

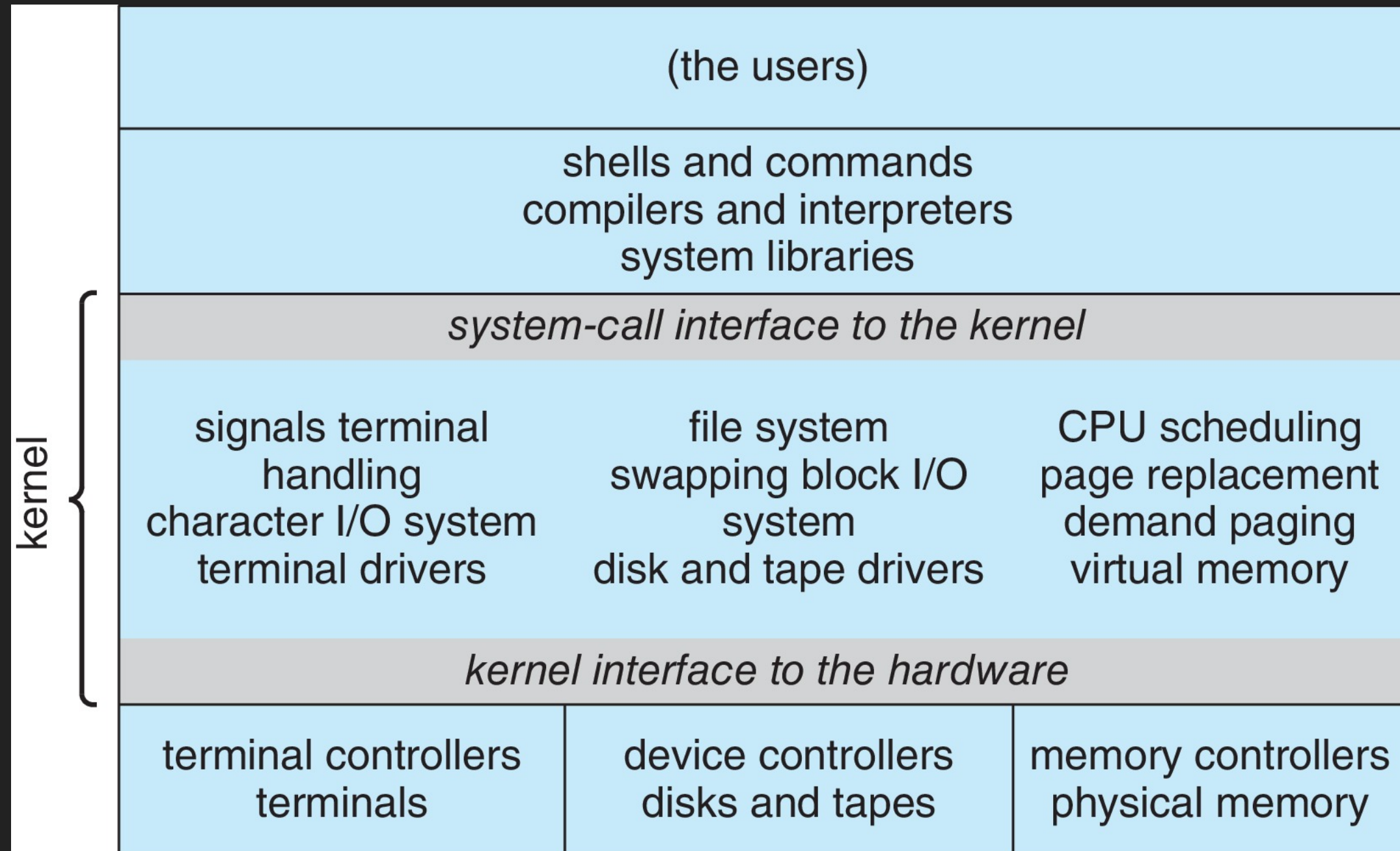
ARCHITEKTUR VON BETRIEBSSYSTEMEN

MONOLITHISCH

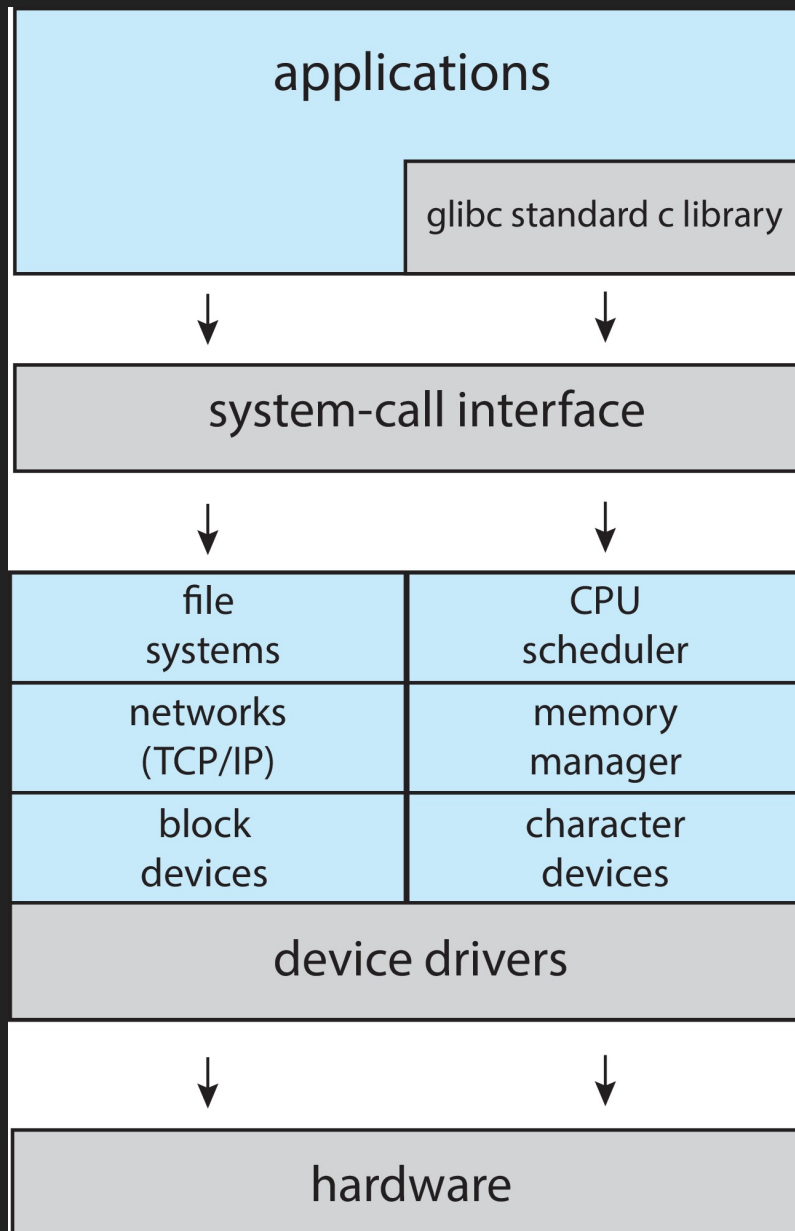


Man spricht hier von einer **monolithischen Architektur**. Der Kernel implementiert alle benötigten Funktionen und stellt diese Anwendungsprogrammen über das system call interface zur Verfügung.

MONOLITHISCH (TRADITIONELLES UNIX SYSTEM)

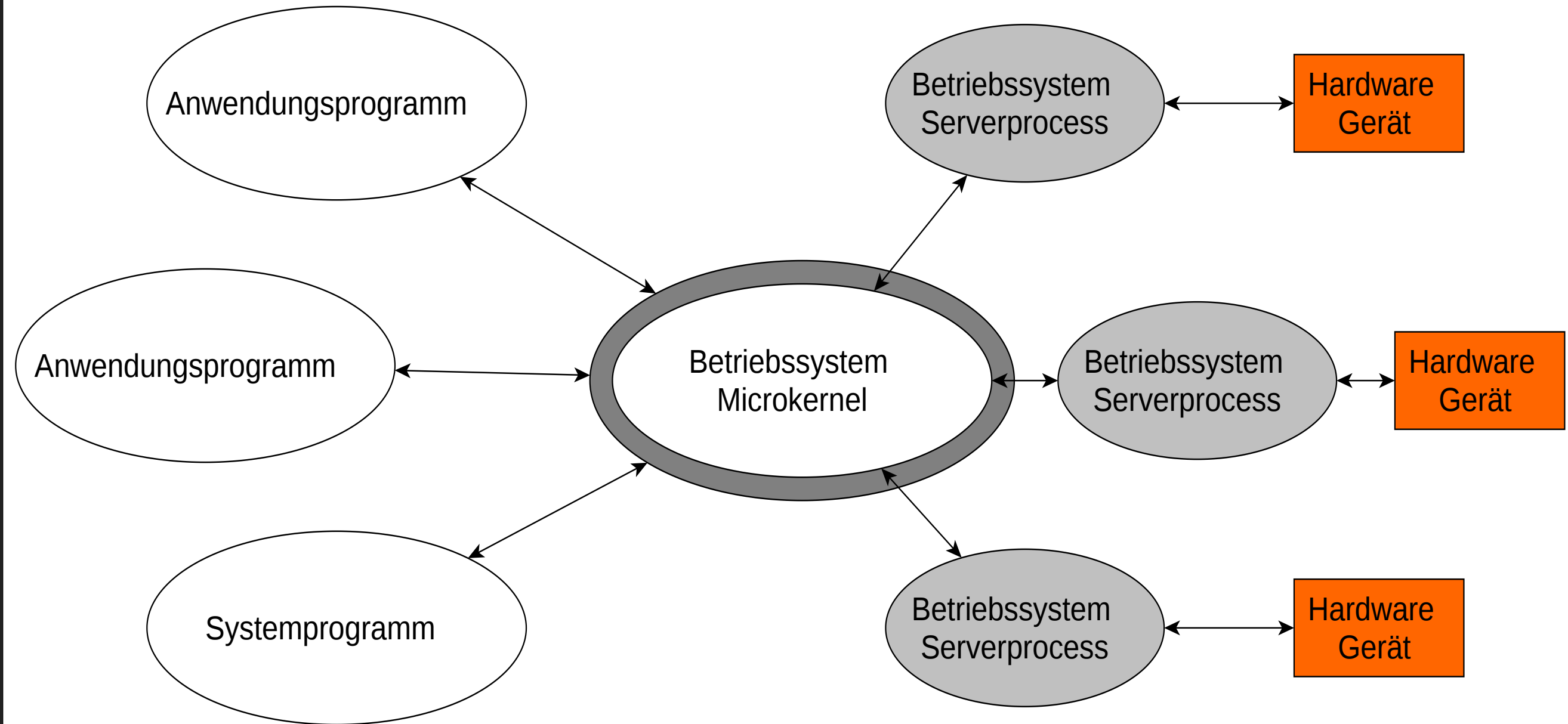


MONOLITHISCH MODULAR (LINUX)

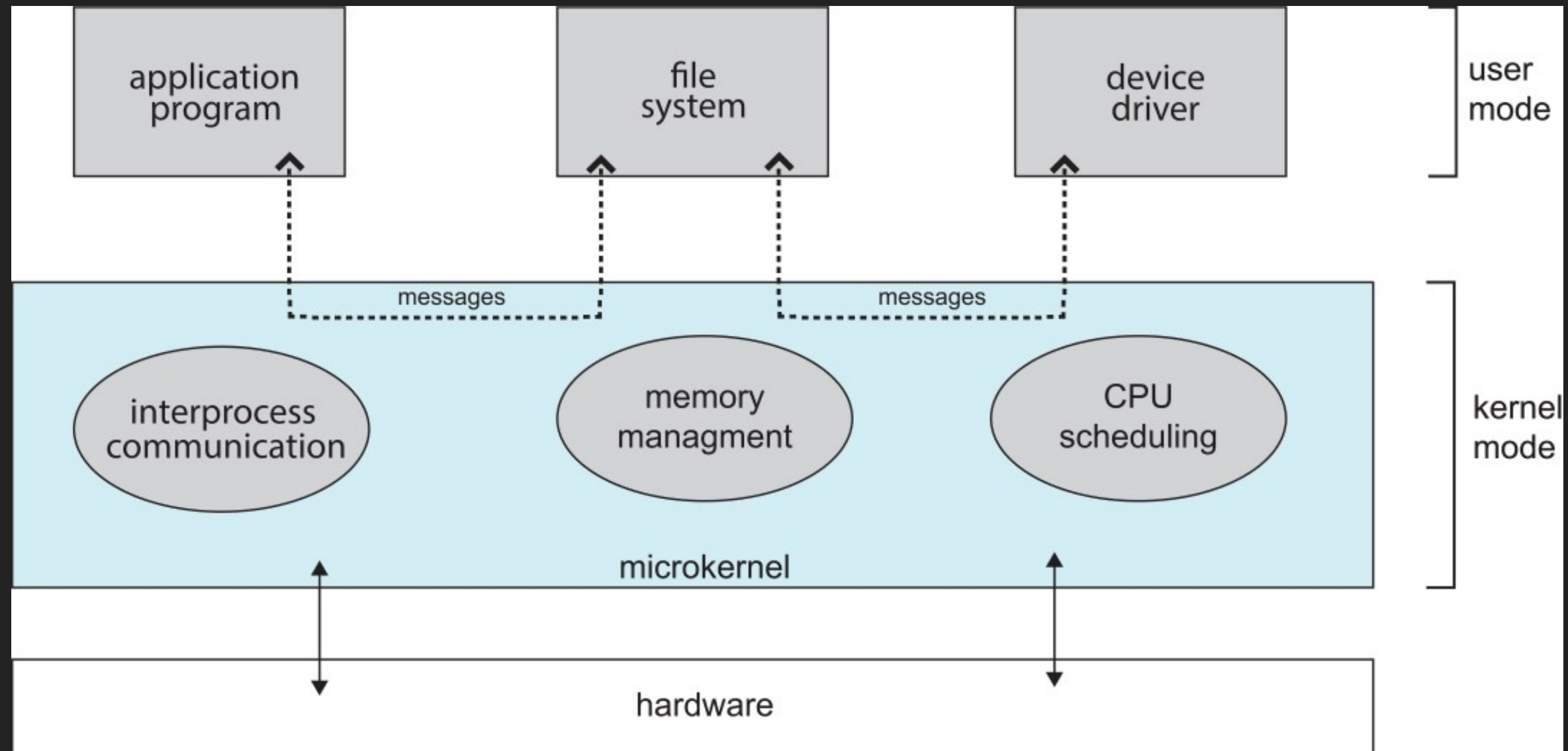


- Linux Systeme verwenden vom Grundprinzip her eine monolithische Architektur
- Verglichen mit der traditionellen Unix Systemen:
 - Stärkere Strukturierung
 - Erweiterbar durch ladbare Kernelmodule (Loadable Kernel Modules (LKM))
 - LKMs erlauben es den Kernel zur Laufzeit mit Funktionalität zu erweitern (z.B. durch laden zusätzlicher Gerätetreiber/Dateisysteme/USB-Geräten)

MICROKERNEL



MICROKERNEL



Das Ziel eines Microkernels ist so viel Funktionalität wie möglich vom kernel mode in den user mode zu verlagern.

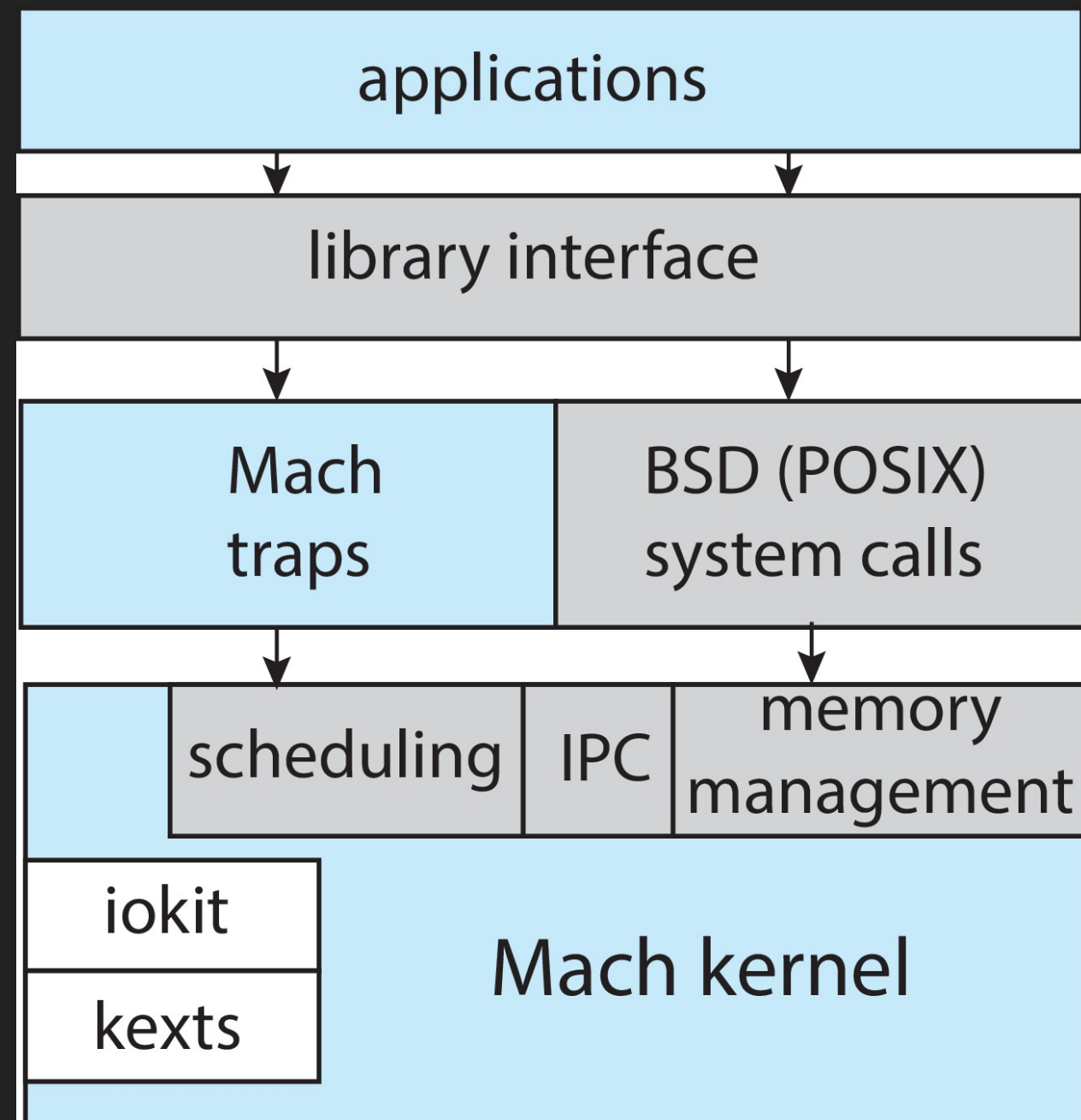
HYBRIDE SYSTEME

AKTUELLE BETRIEBSSYSTEME IMPLEMENTIEREN OFTMALS KONZEPTE VON BEIDEN ARCHITEKTUREN UND WERDEN SOMIT ALS HYBRIDE SYSTEME BEZEICHNET

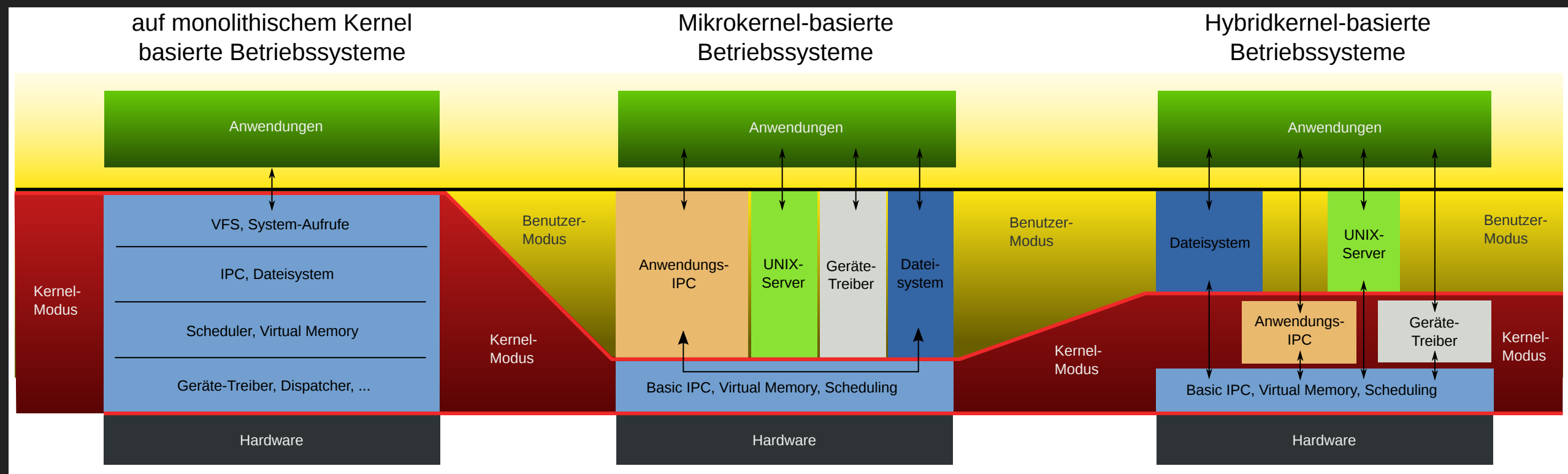
Kein heutiges modernes Betriebssystem wie Windows, MacOS und Linux können strikt einer spezifischen Architektur zugeordnet werden

- **MacOS X** → ist primär Microkernel, sekundär monolithisch
- **Windows** → ist primär monolithisch, sekundär Microkernel
- **Linux** → ist weitestgehend monolithisch, nur kleine Teile implementieren Microkernel-ähnliche Konzepte
 - Bestimmte Subsystem/Treiber im user space (z.B. **FUSE: Filesystem in Userspace**)

HYBRIDE SYSTEME (DARWIN/MACOS)



MONOLITHISCH, MICROKERNEL, HYBRID



<https://de.wikipedia.org/wiki/Datei:OS-structure.de.svg>

EINSCHUB [LINUX LOADABLE KERNEL MODULES]



AUFPASSEN!



EINSCHUB [LINUX LOADABLE KERNEL MODULES]

- Linux ist durch ladbare Kernelmodule **Loadable Kernel Modules (LKM)** erweiterbar
 - LKMs sind Programme mit einer bestimmten und definierten Struktur
 - Sie werden als Teil des Kernels im Kernel space ausgeführt
- LKMs können zur Laufzeit geladen und wieder entfernt werden
- LKMs verwenden primär Datenstrukturen und Funktionen die vom Kernel (oder anderen LKMs) implementiert und bereitgestellt werden.
- Viele Funktionen im kernel space sind sehr ähnlich zu wohlbekannten Funktionen im Userspace

FUNKTIONEN KERNEL VS. USERSPACE

Kernel space	Userspace
<code>printk()</code>	<code>printf()</code>
<code>kmalloc()</code>	<code>malloc()</code>
<code>sprintf()</code>	<code>sprintf()</code>

<https://www.kernel.org/doc/html/docs/kernel-api/>

LOADABLE KERNEL MODULES

Linux Kernelmodule "Hello World!"

```
/*
 * hello-1.c - The simplest kernel module.
 */
#include <linux/module.h> /* Needed by all modules */
#include <linux/kernel.h> /* Needed for KERN_INFO */

int init_module(void)
{
    printk(KERN_INFO "Hello world!\n");

    /*
     * A non 0 return means init_module failed; module can't be loaded.
     */
    return 0;
}

void cleanup_module(void)
{
    printk(KERN_INFO "Goodbye world!\n");
}
```

<https://www.tldp.org/LDP/lkmpg/2.6/html/x121.html>

LOADABLE KERNEL MODULES

hello.c

```
#include <linux/module.h>
#include <linux/kernel.h>

int init_hello_module(void) ❶
{
    printk(KERN_INFO "Hello world!\n"); ❷
    return 0;
}

void exit_hello_module(void) ❸
{
    printk(KERN_INFO "Goodbye world!\n"); ❷
}

module_init(init_hello_module); ❶
module_exit(exit_hello_module); ❸

MODULE_LICENSE("GPL");
```

- ❶ Implementiert und registriert Eingangsfunktion `init_hello_module`
- ❷ Die Funktion `printk` erzeugt Ausgaben im Kernelringpuffer (lesbar mit `dmesg`). `KERN_INFO` ist ein Makro das die Priorität der Ausgabe bestimmt.
- ❸ Implementiert und registriert Exitfunktion `exit_hello_module`

LOADABLE KERNEL MODULES (BUILD)

Makefile

```
obj-m += hello_module.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Build

```
$ make
make -C /lib/modules/4.18.0-0.bpo.1-amd64/build M=/home/ar/lkm/hello modules
make[1]: Entering directory '/usr/src/linux-headers-4.18.0-0.bpo.1-amd64'
CC [M] /home/ar/lkm/hello/hello_module.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/ar/lkm/hello/hello_module.mod.o
LD [M] /home/ar/lkm/hello/hello_module.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.18.0-0.bpo.1-amd64'
$ modinfo hello_module.ko
filename: /home/ar/lkm/hello/hello_module.ko
license: GPL
name: hello_module
vermagic: 4.18.0-0.bpo.1-amd64 SMP mod_unload modversions
modinfo hello_module.ko
```

LOADABLE KERNEL MODULES (LADEN UND ENTFERNEN)

Modulinformation

```
$ modinfo hello_module.ko
filename:      /home/ar/1km/hello/hello_module.ko
license:      GPL
depends:
retpoline:    Y
name:         hello_module
vermagic:     4.18.0-0.bpo.1-amd64 SMP mod_unload modversions
```

Load and Unload

```
$ sudo insmod hello_module.ko
$ sudo rmmod hello_module
$ sudo dmesg -H | tail -2
[Aug26 12:09] Hello world!
[Aug26 12:12] Goodbye world!
```

LOADABLE KERNEL MODULES (ANMERKUNGEN)



- In der zweiten Übung werden wir LKMs für Linux implementieren
 - Bitte verwenden Sie immer zuerst die bereitgestellte oder eine eigene VM um ihren Code zu testen
 - Sie benötigen oft Pakete die normalerweise **nicht** standardmäßig installiert werden
 - Je nach Kernelversion kann es sein das sich Funktionsaufrufe unterscheiden oder deren Definitionen an anderer Stelle befinden
- Grundsätzlich sollten Sie Kernelmodule **immer** zuerst auf VMs ausgiebig testen
 - Bugs, insbesondere beim Speichermanagement, führen dazu das Ihr System sofort einfriert und nicht mehr verwendet werden kann (reboot notwendig)

SYSTEM SERVICES

Systemprogramme bieten eine zweckmässige und benutzerfreundliche Umgebung für Programmentwicklung und -ausführung:

- Dateimanipulation
- Statusinformationen
- Support für Programmiersprachen
- Laden und ausführen von Programmen
- Kommunikation
- Hintergrunddienste
- Applikationsprogramme

Die Benutzersicht auf ein Betriebssystem ist oftmals stark geprägt von Systemprogrammen und nicht durch die eigentlichen system calls.

SYSTEM SERVICES

- Bereitstellung einer benutzerfreundlichen Umgebung für die Entwicklung und Ausführung von Programmen
 - Einige Programme bieten eine sehr einfache Handhabung von Systemaufrufen: z.B. `chmod`, `chown`
 - Andere Programme sind beliebig komplex und bieten weitreichende Funktionalität: z.B. Windows Explorer
- **Dateiverwaltung** - Erstellen, Löschen, Kopieren, Umbenennen, Drucken, Ausgeben, Auflisten und allgemeines Bearbeiten von Dateien und Verzeichnissen
- **Statusinformationen**
 - Systeminformationen: Datum, Zeit, Speicherstatistiken, Speicherplatz
 - Detailinformationen zu: Geschwindigkeit, Logging und Debugging
 - Einige Systeme verwenden eine *Registry* (z.B. Windows) zum speichern und Laden von Konfiguration und Informationen

SYSTEM SERVICES

- **Dateimanipulation**
 - Texteditoren zum Erzeugen und Bearbeiten von Dateien
 - Befehle zum Suchen von Dateiinhalten oder deren Transformation
- **Programmierungsumgebung und Werkzeuge** - Compiler, Assembler, Debugger, Interpreter
- **Laden und Ausführen von Programmen**
 - Linker und Loader: Programme und Mechanismen bereitgestellt durch Systembibliothek und Programme. Z.B. `ld` und `libc` in Linux/Unix
- **Kommunikation**
 - Bereitstellung von Mechanismen um Verbindungen zu anderen Prozessen, Benutzern oder Computersystemen zu ermöglichen

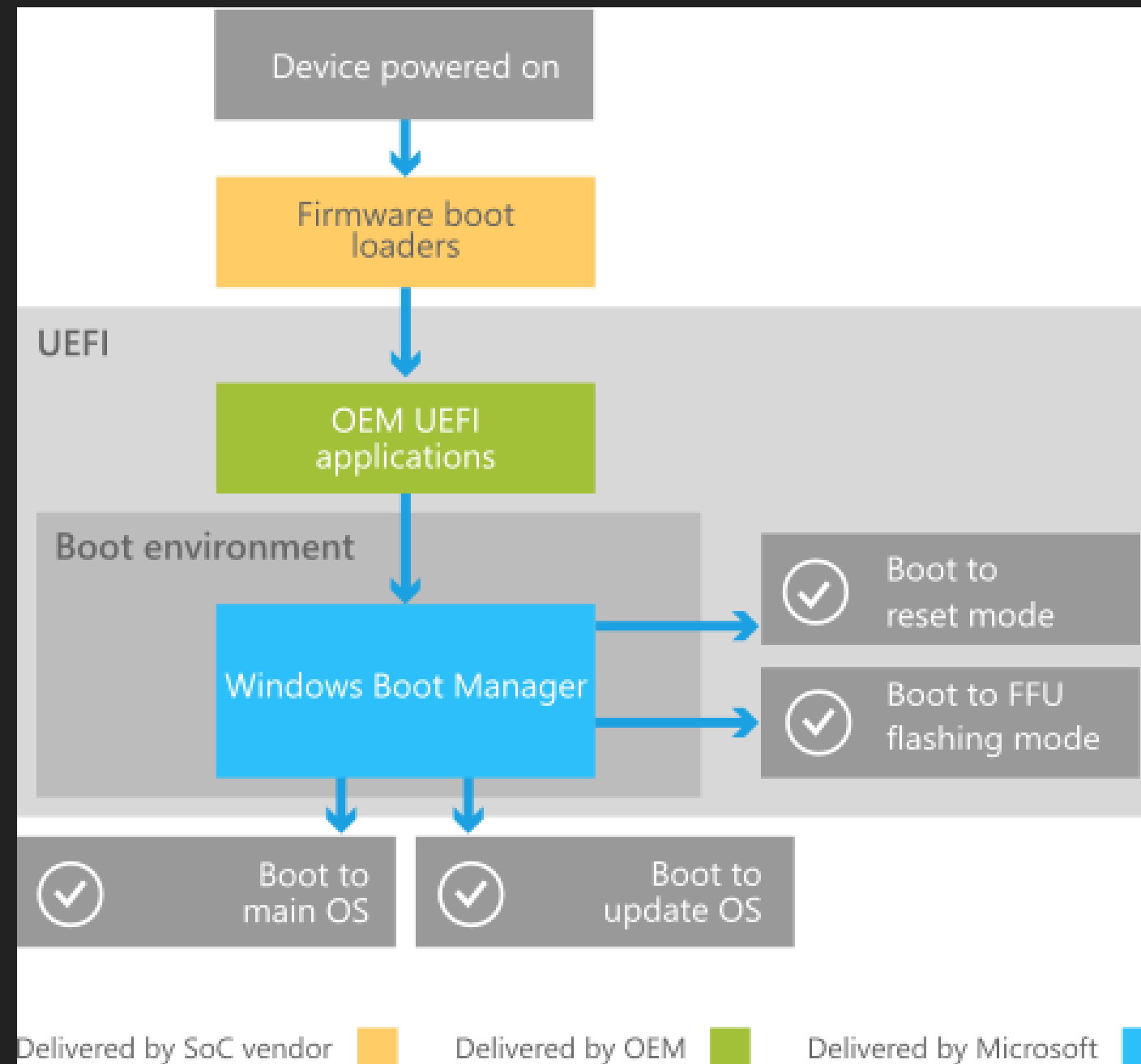
SYSTEM SERVICES

- **Hintergrunddienste** (Background Services)
 - Start oftmals während des Bootvorgangs
 - Einige werden gestartet und wieder beendet: Z.B. Dateisystemcheck
 - Andere laufen von Start bis Reboot des Systems: z.B. Logsystem
 - Bereitstellung von Funktionen wie Festplattenprüfung, Prozessplanung, Fehlerprotokollierung, Drucken, etc.
 - Laufen in Benutzerkontext (user mode) und *nicht* im Kernel
 - Bekannter unter den Namen: *service*, *daemon*
- **Applikationsprogramme**
 - Ausführung durch Benutzer - Kommandozeile, Mausklick
 - Gehören im allgemeinen nicht zum Betriebssystem - Ausnahmen betätigen die Regel :)

START EINES COMPUTERS (BOOTSTRAP)

- Bootstrap-Programm wird beim Einschalten oder Neustart geladen
 - Typischerweise im ROM (Read only Memory) oder EPROM (Eraseable Programmable Read only Memory) gespeichert
 - Früher BIOS (Basic Input Output System), heute oftmals UEFI (Unified Extensible Firmware Interface) oder andere Firmware
 - Initialisiert alle Aspekte des Systems (primär Hardware)
 - Lokalisiert das Gerät das den Bootloader enthält
 - Lädt den Bootloader und gibt die Kontrolle ab

START EINES COMPUTERS (UEFI WINDOWS)



START EINES COMPUTERS (BOOTLOADER UND KERNEL)

→ BOOTLOADER HAT DIE KONTROLLE ←

- Auffinden den Betriebssystemkerns
- Laden des Betriebssystemkerns (Kernel)
- Abgabe der Kontrolle an Kernel
 - Ab hier wird es spezifisch je nach verwendetem Betriebssystem.

START EINES COMPUTERS (LINUX)

→ **KERNEL HAT DIE KONTROLLE** ←

- Laden von zusätzlichen Gerätetreibern
- Selbstinitialisierung des Kernels
 - Anpassung an Hardware (CPU/MMU)
 - Speichermanagement (heap/stack)
 - Scheduler
 - Interrupts
- Starten des ersten Prozesses (früher `init` heute oft `systemd`)



- Historisch interessant: [Linux V1.0 Bootcode](#)
- Heute: <https://github.com/torvalds/linux/blob/master/init/main.c>

NACHBEARBEITUNG: AUFGABEN UND FRAGEN

BETRIEBSSYSTEME EINFÜHRUNG

- Nennen Sie 3 Hauptmerkmale die ein Betriebssystem bereitstellt
- Unter welcher Voraussetzung kann auf den Einsatz eines Betriebssystems verzichtet werden?

INTERRUPTS

- Erläutern Sie kurz und in eigenen Worten um was es sich bei einem Interrupt handelt?
- Welcher Hardwarekomponente und welche Softwarekomponente sind primär zuständig für die Behandlung von Interrupts
- In welchem Zusammenhang steht ein Hardware und ein Softwareinterrupt?
- Nennen Sie 2 Beispiele von Hardwareinterrupts und Softwareinterrupts
- Ein Softwareinterrupt kann auch einem Benutzerprozess ausgelöst werden. Erläutern Sie kurz warum dies sinnvoll ist.
 - Gehen Sie hierbei auch auf Sicherheitsaspekte ein. Was wird hier eigentlich voneinander isoliert?

INPUT/OUTPUT

- Erläutern Sie kurz und in eigenen Worten den Unterschied zwischen blockierender und nicht-blockierender I/O

COMPUTERARCHITEKTUR

- Erklären Sie anhand von Diagrammen den Aufbau eines Computersystems hinsichtlich folgender Komponenten:
 - Prozessor, CPU, Register, Cache, Hauptspeicher
 - Unterscheiden Sie ferner wie sich der Aufbau für mehrere (1) Single-Core und (2) Multi-Core Systeme unterscheidet

SPEICHERGERÄTE

- Sortieren Sie die folgenden Speichergeräte Aufsteigend ahnadh deren Zugriffsgeschwindigkeit (schnellste steht oben)
 - Festplatten, Cache, NVME (SSDs), Magnetische Bänder, Register, Cache, RAM (Hauptspeicher), Optische Laufwerke
- Was ist ein weiteres Kriterium für den Einbau von bestimmten Speichergeräten außer der Geschwindigkeit?

BETRIEBSMODI

- Erläutern Sie kurz die Begriffe **User Mode** und **Kernel Mode**
- Wie werden die Modi auf Hardwareebene unterschieden?
- Zeichnen Sie ein Diagramm das den Wechsel zwischen User Mode und Kernel Mode beschreibt
- Der Wechsel zwischen beiden Modi wird von welcher speziellen Art von Befehlen ausgeführt?

DIENSTE UND INTERFACES

- Nennen Sie 3 Dienste die von einem Betriebssystem zur Verfügung gestellt werden und erläutern Sie kurz worum es sich bei dem Dienst handelt
- Welche Art von Interfaces zur Bedienung eines Rechners kennen Sie
- Geben Sie zu folgenden Betriebssystemen die primäre Art des Interfaces an. Begründen Sie kurz Ihre Aussage falls es nicht eindeutig zugeordnet werden kann
 - Linux
 - MACOSX
 - Windows

SYSTEM CALLS

- Wozu dienen Systemaufrufe (System Calls)?
- Nennen Sie 3 Ihnen bekannte System Calls und erläutern Sie kurz was diese System Calls tun.
- Worin unterscheidet sich der Aufruf die Verwendung der Funktion `printf("ABC\n");` und der Funktion `syscall(SYS_write, 1, "ABC\n", 4);`
 - Bei was handelt es sich um eine normale Funktion und bei was um einen Systemaufruf?

SYSTEM CALLS 2

Betrachten Sie folgenden Assemblercode und beantworten Sie die Fragen zu den Punkten unterhalb des Codeabschnitts.

```
.text
ENTRY (syscall)
    movq %rdi, %rax    1
    movq %rsi, %rdi    2
    movq %rdx, %rsi    2
    movq %rcx, %rdx    2
    movq %r8, %r10     2
    movq %r9, %r8       2
    movq 8(%rsp), %r9   3
    syscall
    cmpq $-4095, %rax
    jae SYSCALL_ERROR_LABEL
    ret
PSEUDO_END (syscall)
```

- 1 Was macht diese Zeile?
- 2 Was machen diese Zeilen?
- 3 Was macht diese Zeile?

ARCHITEKTUR VON BETRIEBSSYSTEMEN

- Erläutern Sie kurz die Begriffe Microkernel, Monolithischer Kernel und Hybrider Kernel?
- Warum ist es bei heutigen Systemen schwer genau zu bestimmen um welche konkrete Form der Architektur es sich bei einem Betriebssystem handelt?

LOADABLE KERNEL MODULES (LKM)

- Was sind sog. Loadable Kernel Modules und warum verwendet Linux LKMs?
 - Welche Art von Aufgaben eignen sich als LKM implementiert zu werden und welche eher nicht?
 - Auf was sollte bei der Entwicklung von LKMs besonders geachtet werden? Kann hier unter Umständen etwas kaputt gehen?

BOOTPROZESS

- Erläutern Sie kurz und in eigenen Worten wie der Bootprozess abläuft. Fangen Sie mit Ihrer Erläuterung beim "anschalten" der Rechners an
- Ab welchem Zeitpunkt ist das Betriebssystem vollständig gestartet?
 - Lässt sich der Zeitpunkt eindeutig bestimmen?